

Nikah Platform

Complete Software Design Document

Mobile-First (Kotlin/Android) · Modular Monolith · Self-Hosted

Document version	1.3
Companion documents	Architecture Design Document v3.4 (Appendix A); OpenAPI Specification v0.5
Primary client	Android app, Kotlin, native
Target scale	Up to 1,00,000 (1 lakh) users
Hosting	Self-hosted VPS (India region), Docker
Regulatory scope	India (DPDP Act 2023) — primary; other jurisdictions deferred

This document consolidates every decision made across the requirements, database, Wali-authority-model, and API design phases into the full structure below. It supersedes no prior document — the detailed Entity Catalog, Attribute Classification, Relationship Matrix, and ER Diagram live in the companion Architecture Design Document v3.4, reproduced in full as Appendix A rather than duplicated here.

Table of Contents

Version History

1. Introduction
 2. Product Overview
 3. Functional Requirements
 4. Business Rules
 5. Non-Functional Requirements
 6. User Roles
 7. Domain Model
 8. C4 Architecture
 9. Functional Modules
 10. Database Design
 11. API Design
 12. Security Architecture
 13. Infrastructure & Deployment
 14. Architecture Decisions (ADRs)
 15. Testing Strategy
 16. Future Enhancements
- Appendix A — Full Entity Catalog & ER Diagram (v3.4)
- Appendix B — Open Issues
- Appendix C — Glossary
- Appendix D — Reference Documents

Version History

Version	Date	Changes
1.0	2026-07-10	Initial complete design document: Sections 1–16 written from the settled requirements, database design (v3.3), and API design (Sprint 4/OpenAPI v0.4). C4 Context and Container diagrams, ER diagram, and a deployment diagram included. Target scale 10,000–70,000 users. Terminology: “Guardian” used as the entity/field name throughout.
1.1	2026-07-13	Sequence diagram for the Wali-approval-and-messaging flow added (Section 8.1), rebuilt with a more robust lifeline layout after the first attempt didn't render cleanly. Voice-calling design question resolved: an active, Proxy-control Wali may join a voice call (Section 16). Target scale increased to up to 1,00,000 (1 lakh) users, updated everywhere it's referenced (NFRs, ADR-005/006, caching rationale, load-test targets). Terminology swept throughout: “Guardian” → “Wali” in all entity names, field names, URL paths, error codes, and prose, across this document, the companion Architecture Design Document, the OpenAPI specification, and the Sprint 4 API design notes.
1.2	2026-07-13	Major infrastructure decision: backend migrated to self-hosted Supabase (Docker, same VPS — ADR-005 preserved, not reversed) for Auth (phone-OTP, replacing custom User/Session/OTP), Storage, and Realtime (replacing the custom Socket.IO service, ADR-003 superseded by new ADR-009). Business logic deliberately kept in Next.js, not Supabase RLS/Edge Functions (ADR-010). C4 Container and Deployment diagrams rebuilt; Sections 8, 10, 11, 12, 13 and the ADRs (14) updated; Architecture Design Document bumped to v3.4; OpenAPI specification bumped to v0.5 (24 endpoints, auth endpoints removed). Two previously open items resolved: Address stays structured (confirmed), MobileNumber encryption is now moot (Supabase Auth's responsibility).
1.3	2026-07-13	Five PlantUML diagrams added, generated from companion .puml scripts (renderable at plantuml.com) and verified locally before inclusion: a new Use Case Diagram (Section 6.1) and Domain Model Class Diagram (Section 7) supplement the existing tables; the Container, Sequence, and Deployment diagrams (Figures 4, 5, 6) were replaced with cleaner PlantUML-rendered versions of the same content, superseding the earlier hand-built diagrams. All figures renumbered sequentially (1–6).

A Supabase backend-migration analysis is in progress as of v1.1 (see chat discussion) — pending answers on Auth strategy, hosting model, Realtime provider, and business-logic placement before Sections 8, 10, 12, 13, and the ADRs are updated to reflect a decision.

1. Introduction

Purpose. This document is the complete design reference for the Nikah Platform, from product intent through to deployment and testing. It is written for three audiences at once: the founder directing development, any future engineer joining the project, and AI coding agents (Claude Code) that need durable, reviewable context rather than re-derived assumptions.

Scope. Covers the mobile-first (Kotlin/Android) matrimony platform: profile creation, Wali-mediated matching and messaging, verification, subscriptions, and moderation. Web and iOS are explicitly Phase 2+ (Section 16).

How this document was built. Every business rule in Sections 3–4 traces back to a specific, resolved decision from the requirements, database-design, and Wali-authority-model review sessions, and from two live UI audits of the working prototype (react-app-beta-ruby.vercel.app). Nothing here is a first guess presented as final — open items are isolated in Appendix B rather than silently decided.

2. Product Overview

Vision. “Your Nikah begins with sincerity.” A Sharia-compliant matrimony platform where family involvement (the Wali) is a first-class part of the product, not a bolted-on afterthought — privacy, verification, and Wali oversight are structural, not optional settings.

Target market. India first (up to 1,00,000 (1 lakh) users targeted for initial scale), with the platform's data model already built to extend to the Gulf/South Asian diaspora (UAE, UK, USA, Canada, Pakistan, Bangladesh, etc.) once India-first validation succeeds.

Key differentiators:

- Wali as a structural role, not an add-on — with real authority (approve/reject, contact disclosure) enforced server-side.
- A dual-mode Wali model: full proxy control for a registered relative, or simple display-only listing for notifications only.
- Verified profiles and a directional, trust-blended matching engine.
- Instant messaging (WhatsApp-like delivery), gated by family approval, not open by default.
- A free-form Quick-Match Wizard teaser before full registration, to reduce signup friction.

3. Functional Requirements

Module	Functional Requirements
Authentication	Register/login via OTP (phone or email). Session management. Login audit trail.
Profile	Create/edit core profile, religious preferences, marriage intentions, background details (children, disability, languages, hobbies). Enforce completion-percentage gate for search visibility.
Wali	Register as proxy Wali or list a display-only Wali. Request promotion (candidate-approved) or swap (Admin-approved). Approve/reject conversations. Emergency manual contact-share.
Partner Preference	Capture match criteria (age, sect, madhhab, religiosity, marital status, location, living arrangement, etc.).
Quick-Match Wizard	Lightweight pre-registration teaser: 8-step bilingual (English/Hindi) flow producing a preview match list before full signup.
Verification	Submit and review phone/identity/photo verification; maintain a trust score. No ID verification required for Walis in MVP.
Matching	Generate directional, trust-blended match candidates per profile.
Search	Filter/sort/paginate profiles meeting the completion and visibility gates.
Messaging	Wali-gated conversations; instant (real-time) message delivery; per-side routing to whichever account currently controls that side.
Subscription & Payment	Plans, Razorpay (India) / Stripe (international) checkout, gender/age/marital-status discount rules, webhook-driven activation.
Media	Upload up to 4 photos and one optional short video introduction; moderation gate before visibility.
Privacy	Per-profile visibility controls for photos, contact, and profile; female-restrictive defaults.
Notification	Push notifications (FCM) for matches, messages, verification status, and Wali-related events.
Reporting/Moderation	File and review reports against profiles or messages.
Administration	Role-based (Admin/Moderator) management of verification, subscriptions, and reports.

4. Business Rules

Full detail and audit trail in the Architecture Design Document v3.4 (Appendix A). Summarized here:

- **Profile completeness gate:** $\geq 60\%$ completion required before Search/Matching visibility.
- **Female privacy default:** female profiles default to private visibility; stricter photo/contact defaults.
- **Signed-URL-only media:** Photo and Video are never served from a public path.
- **Subscription discounts (men only):** single highest-eligible discount (age >55 , divorced, widowed, revert) — never stacked. Women's accounts are free.
- **Wali authority model:** exactly one Wali slot per profile; VerificationLevel and ControlLevel tracked independently; only a Proxy-control Wali (or the candidate herself, if none) can approve/reject conversations or trigger contact disclosure.
- **Wali change:** Promotion (display-only $\rightarrow$ Proxy) is candidate-approved; Swap (replace the Wali) is Admin-approved and requires a stated reason.
- **Per-side conversation control:** the moment a side's Proxy Wali approves, that side's messaging is handled exclusively by the Wali's account — the candidate is excluded from that specific conversation only.
- **Contact disclosure is an audited event:** every reveal of contact info (via Wali approval, manual emergency share, or no-Wali self-approval) is logged with who/why/when.
- **Directional, trust-blended matching:** match rows are generated per-profile; BlendedScore combines compatibility with live trust score.
- **Fields never collected:** sexual history, exact home address, credit score/debt, social media credentials, political affiliation, forced medical/immigration disclosures, unwarranted criminal history, photos of children, mandatory skin colour, IQ results, menstrual/reproductive medical details.
- **No ID verification required for Walis** in the MVP — accepted risk, see ADR-007.
- **Instant delivery is a hard requirement** for messaging (not best-effort) — see Section 8/13 for the WebSocket architecture this implies.

5. Non-Functional Requirements

Category	Requirement
Scale	Up to 1,00,000 (1 lakh) registered users targeted. Assume ~5–10% concurrent at peak (5,000–10,000 concurrent sessions) — single well-specified VPS is adequate at this range; see ADR-005/006 for the growth path.
Availability	Target 99.5% uptime for Phase 1 (self-hosted, single VPS with monitoring and automated restart, not yet multi-region failover).
Performance	API p95 latency < 300ms for standard reads/writes. Message delivery end-to-end < 2 seconds (instant-delivery requirement) via the Realtime Messaging Service, not REST polling.
Security	All traffic over TLS. Session tokens expire and are checked server-side on every request. Signed URLs for all media. Payment webhooks signature-verified (Razorpay/Stripe).
Privacy	DPDP Act 2023 (India) is the primary regulatory target for Phase 1. Female-restrictive privacy defaults. Sensitive-field classification per Attribute Classification (Appendix A).
Localization	English and Hindi supported from the Quick-Match Wizard onward; full bilingual coverage is a Phase 1 goal, not deferred.
Platform coverage	Native Android (Kotlin) first. Full website second (Phase 2). iOS deferred (Section 16).
Maintainability	Modular monolith with clear module boundaries (Section 9) so any module can be extracted into its own service later without a rewrite.

6. User Roles

Role	Description
Bride / Groom (Candidate)	Self-registered profile owner, or a profile registered on her behalf by a Proxy Wali.
Wali — Proxy	Registered account holder with full control over a profile: edit, browse, approve/reject conversations, receive contact disclosures. Can manage multiple candidates' profiles from one account.
Wali — Display-only	Listed by name/relation only, no account, no control — receives notifications only (new match, conversation started, marriage-stage progress).
Admin	Full administrative permission (User.Role) — verification, subscriptions, reports, Wali Swap approval.
Moderator	Reviews reported profiles/messages; escalates serious cases to Admin.
Payment Gateway (Razorpay/Stripe)	External system processing subscription payments.
SMS/Email Provider	External system delivering OTPs and notifications.
FCM	External system delivering push notifications to the Android app.

6.1 Use Case Diagram

Every role's interactions with the platform, including the two Wali change flows and the server-side authority check that gates conversation approval — see the notes on the diagram itself.

Nikah Platform — Use Case Diagram

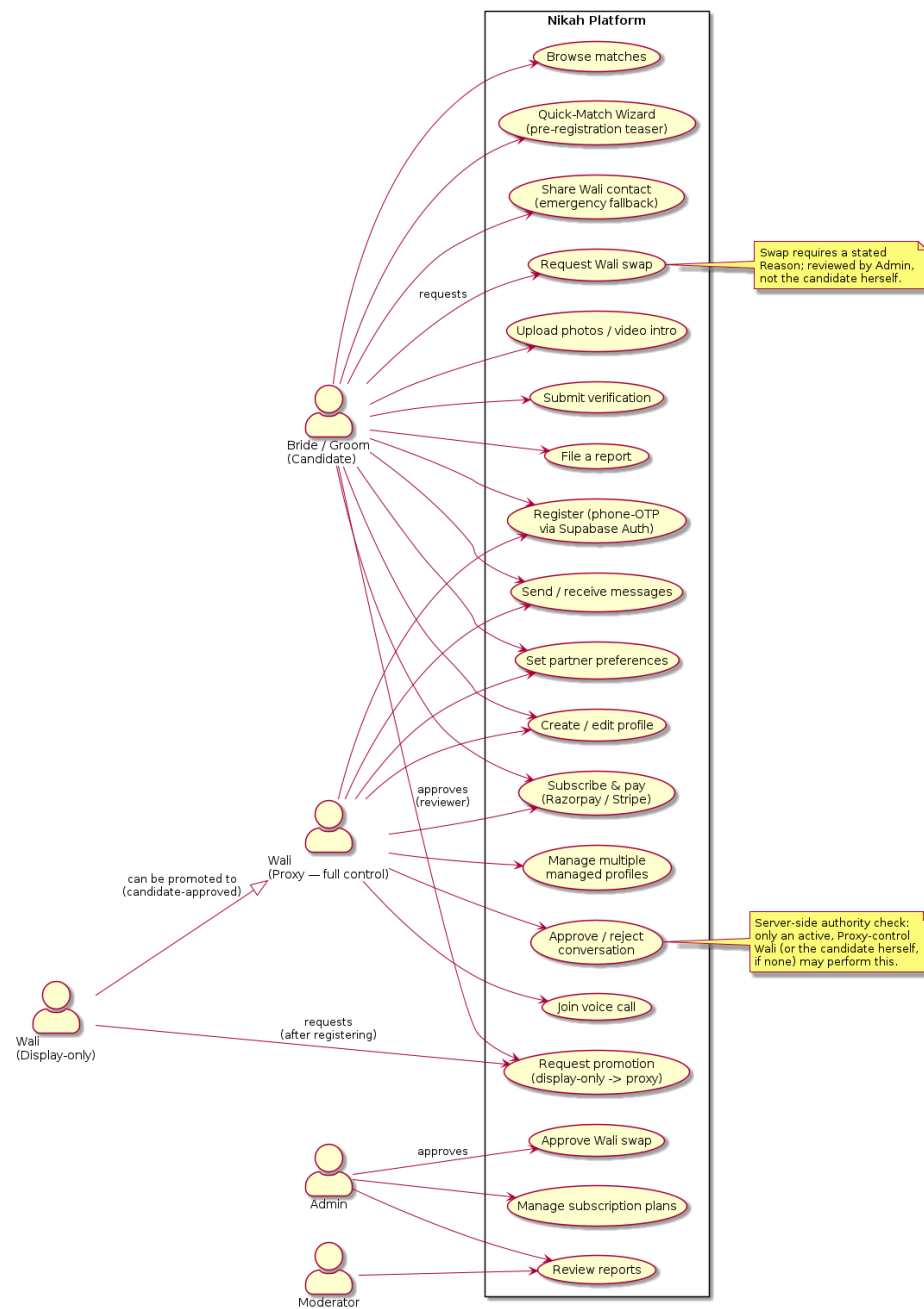


Figure 1 — Use Case Diagram (PlantUML)

7. Domain Model

The full Entity Catalog (32 entities), Attribute Classification, and Relationship Matrix are reproduced in **Appendix A** from the Architecture Design Document v3.4 — not duplicated here to avoid two sources of truth drifting apart.

Entities, grouped by module:

Module	Entities
Authentication	User (thin extension of Supabase auth.users), LoginLog
User Profile	Profile, ReligiousPreference, ProfileLifestyleGoal, ProfileWeddingRitual, MarriageIntention, ProfileChildrenDetail, ProfileDisabilityType, ProfileLanguage, ProfileHobby, Education, Occupation, Address
Partner Preference	PartnerPreference
Quick-Match Wizard	QuickMatchPreference
Wali	Wali, WaliChangeRequest, ConversationApproval, ContactDisclosure
Verification	VerificationRequest
Matching	MatchCandidate
Messaging	Conversation, Message, Attachment
Subscription	SubscriptionPlan, Subscription, Payment
Media	Photo, ProfileVideo
Privacy	PrivacySetting
Reporting/Moderation	Report

The full ER diagram (attribute-level detail, PK/FK types) appears in Appendix A. The class diagram below is a companion view, colour-coded by module, showing the same 32 entities and their relationships in UML class-diagram notation.

8. C4 Architecture

Level 1 — System Context and **Level 2 — Container** diagrams follow on the next two landscape pages. Three architecture decisions drive their shape: the primary client is a **native Kotlin Android app**; messaging must be **instantly delivered**; and (v1.2) the backend now runs on a **self-hosted Supabase stack** for Auth, Storage, and Realtime, while **Next.js remains the sole place business rules are enforced**.

What Supabase replaced: the custom Socket.IO + Redis Realtime service from v1.1 is gone — **Supabase Realtime** now handles instant message push, broadcasting Postgres changes to subscribed clients. The custom User/Session/OTP entities are gone too — **Supabase Auth (GoTrue)**, self-hosted, owns phone-OTP login and issues the JWTs the API Backend verifies on every request. Generic S3-compatible storage became **Supabase Storage**, which is S3-compatible under the hood anyway, so nothing about the signed-URL design in Section 12 had to change.

What did not change, deliberately: POST /conversations/{id}/messages (Section 11) remains the only place a message can be written, and the only place the Wali-approval business rule is enforced. Supabase Realtime's job is purely to notice that write and fan it out — and a Row-Level Security policy checking ConversationApproval/*ManagedByWaliId gates who can even subscribe to a conversation's changes, acting as a second, independent, database-level check underneath the application-level one. This is a genuine security improvement over the v1.1 design, not just a simpler ops story: even a bug in the Next.js code couldn't leak a message to an unauthorized subscriber, because the database itself won't hand over the row.

What deliberately stayed out of Supabase: Wali-approval branching, contact disclosure, gateway selection, and discount calculation all remain Next.js application code, not Postgres RLS policies or Supabase Edge Functions (confirmed decision, ADR-009) — that logic is too conditionally complex to safely split across two systems and two languages (SQL policies vs. TypeScript).

v1.3: the Container, Sequence, and Deployment diagrams (Figures 4, 5, 6) are now rendered from PlantUML source (companion .puml files, renderable at plantuml.com or any local PlantUML install) rather than the earlier hand-built diagrams — standard UML notation, easier to hand-edit going forward. The Context diagram (Figure 3) remains as originally built, since no PlantUML equivalent was requested for it.

C4 Model \u2014 Level 1: System Context

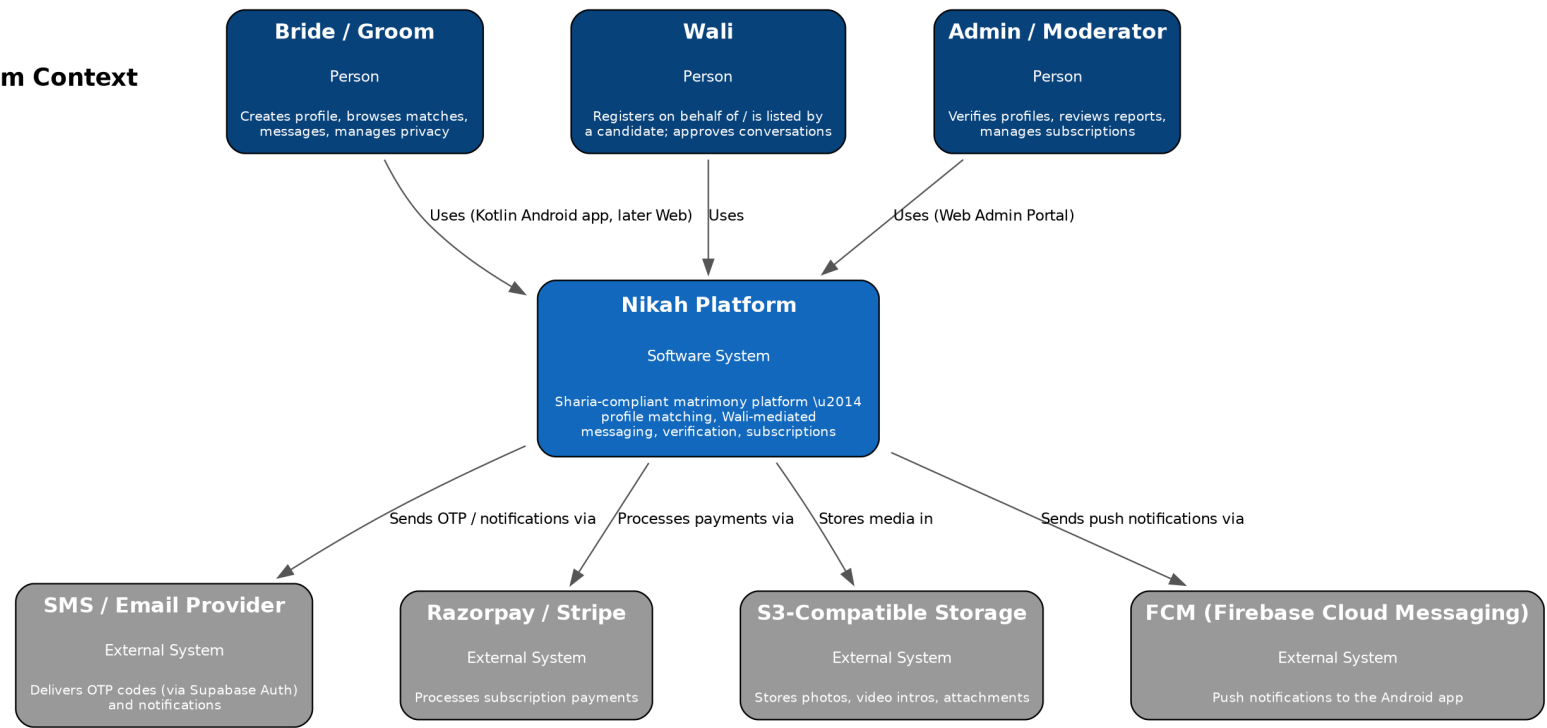


Figure 3 — C4 Context Diagram

Nikah Platform — Container Diagram (C4 Level 2 style, v1.2)

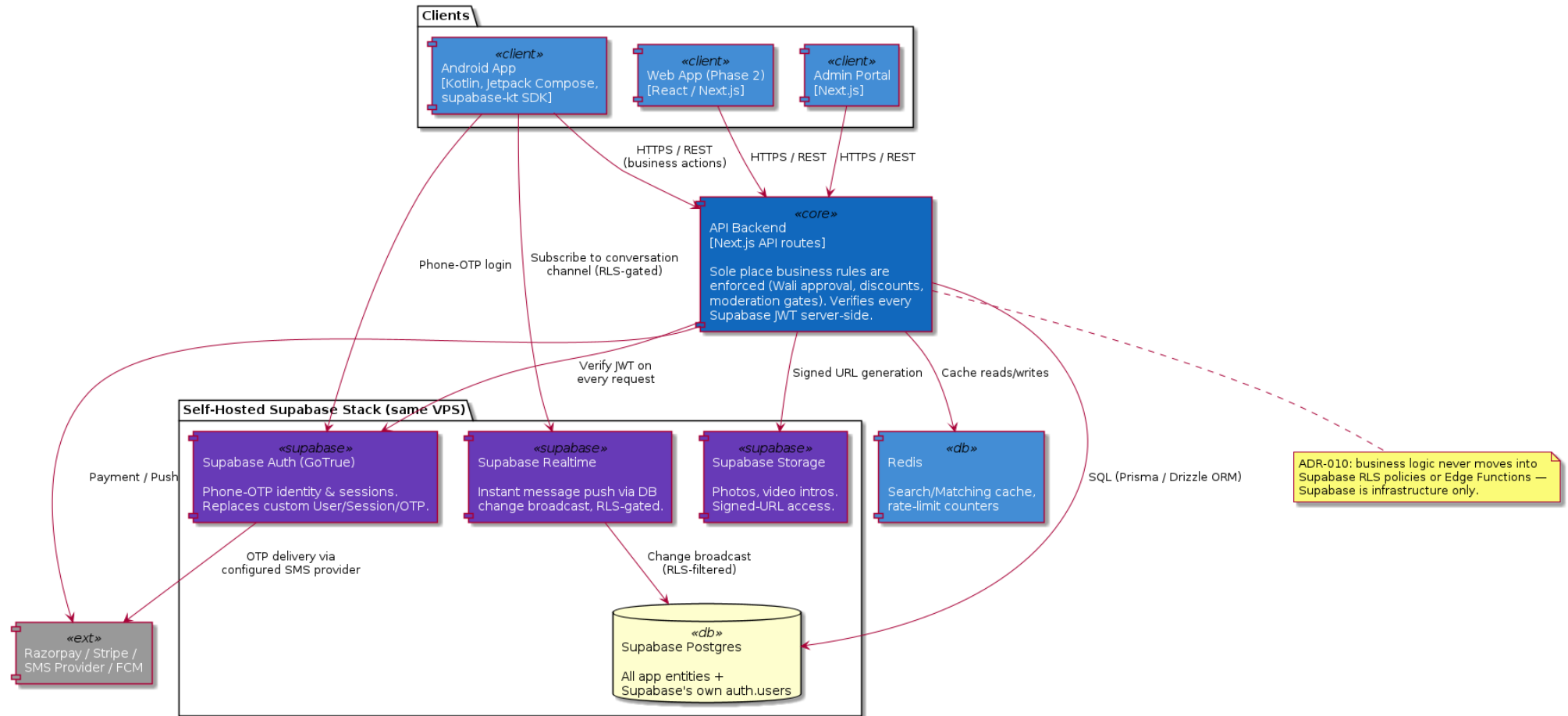


Figure 4 — C4 Container Diagram (PlantUML, self-hosted Supabase stack)

8.1 Key Flow — Wali Approval and Instant Messaging

The sequence diagram below walks through the single flow that exercises nearly every business rule in this document at once: a suitor's interest, the Wali's approval, contact disclosure, and the resulting instant message delivery being routed to the Wali's device rather than the candidate's own, per Section 4's per-side control rule. The “Database” and “Realtime Service” lifelines below now map to Supabase Postgres and Supabase Realtime respectively — the flow of responsibility is unchanged from v1.1, only the infrastructure underneath it.

Nikah Platform — Interest → Wali Approval → Instant Messaging

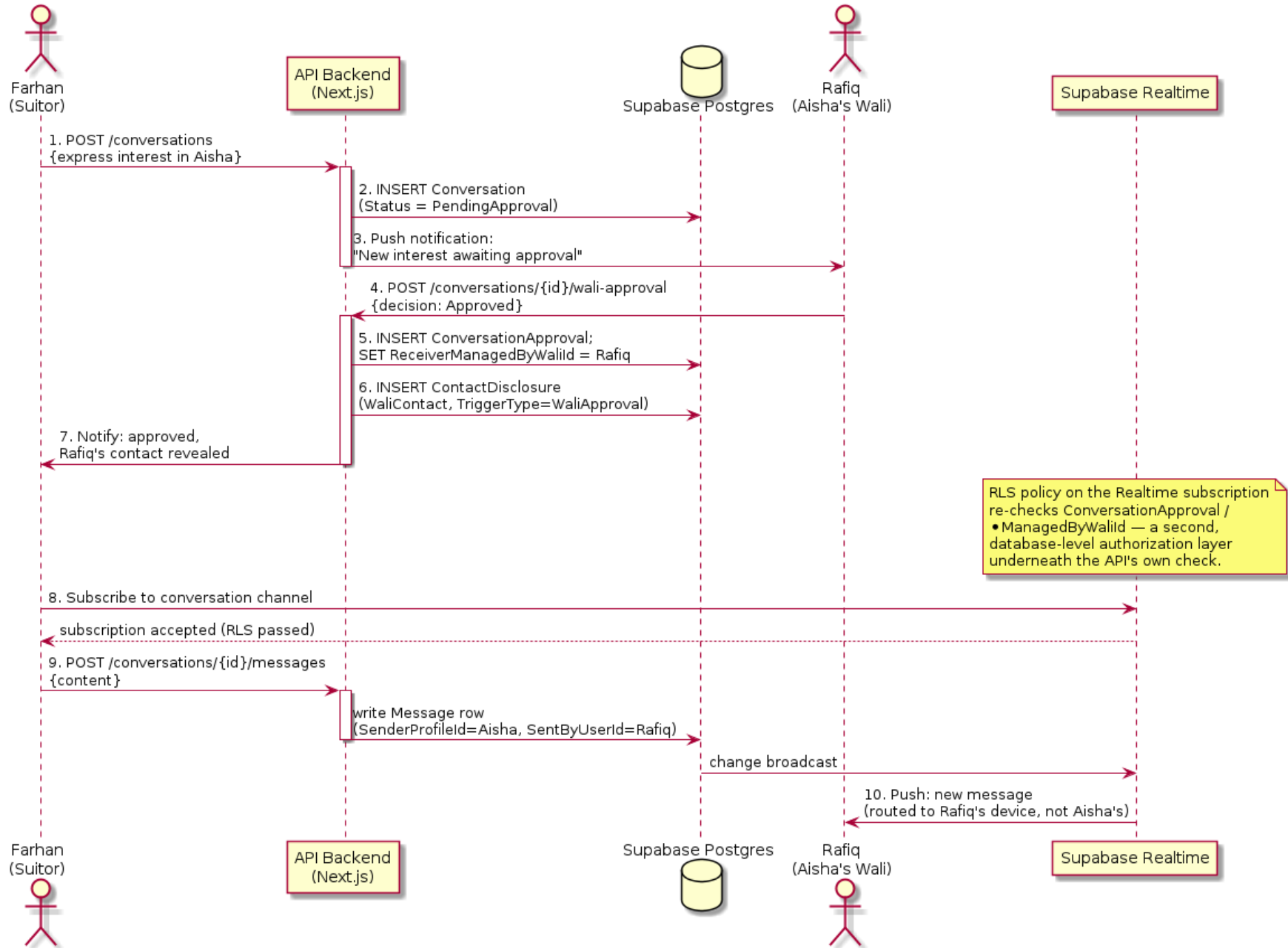


Figure 5 — Sequence Diagram: Interest → Wali Approval → Instant Messaging (PlantUML)

9. Functional Modules

Module boundaries (full responsibilities and owned entities in Appendix A, Section 3) are unchanged by the mobile-first pivot — the API backend remains a modular monolith regardless of which client consumes it. Two additions:

Module	Addition
Messaging	Now spans two containers: the REST API (writes, history) and the Realtime Messaging Service (instant push delivery).
Notification	Now concretely FCM-based for the Android app; push tokens tracked per-device, not just per-user.

10. Database Design

Full Entity Catalog, Attribute Classification, and Relationship Matrix: **Appendix A**. Schema design notes specific to the confirmed 1-lakh-user scale target:

- **Indexing:** composite indexes on Profile(Country, Sect, Gender, VisibilityStatus) for Search; index on Message(ConversationId, SentAt) for fast history reads; unique index on Payment.GatewayTransactionId.
- **Address stays structured** (City and State as separate fields, not collapsed to free text) — confirmed decision.
- **Read-heavy tables** (Profile, MatchCandidate) are the primary target for Redis caching at this scale — not because 1 lakh strictly requires it, but because Search/Matching queries are the most repeated read pattern.
- **No sharding or read-replica requirement** at this scale — a single well-resourced Postgres instance on the VPS is sufficient for Phase 1 (see ADR-005).
- **ORM:** Prisma or Drizzle, connecting to Supabase's Postgres exactly as it would to any Postgres instance — Supabase adds no proprietary query layer our ORM has to route through. Drizzle preferred if the Matching module's blended-score queries need close-to-SQL control.
- **v1.2 — identity now lives partly outside our own schema:** User is a thin extension table (Role, Status only), keyed to Supabase's own auth.users.id. Every FK that used to point at our custom User table (Profile.UserId, Guardian.UserId, etc.) is unchanged in shape — it just now ultimately traces back to a Supabase-managed identity rather than a row we created ourselves via a custom OTP flow.
- **Row-Level Security (RLS):** used as a second, defense-in-depth authorization layer specifically for Supabase Realtime subscriptions (Section 8) and Storage access — never as the primary place a business rule is decided. All authorization logic still originates in, and is owned by, the Next.js API layer (ADR-009).

11. API Design

Full OpenAPI 3.0 specification (24 endpoints, validated with openapi-spec-validator): companion file *Nikah_Platform_OpenAPI_v0.5.yaml*. Summary by module:

Tag	Endpoints	Notes
Auth	<i>None — not a REST endpoint of ours.</i>	v1.2: phone-OTP is handled entirely by the Supabase Auth client SDK (supabase-kt) talking directly to self-hosted Supabase Auth. This backend verifies the resulting JWT on every request instead.
Profile	/profiles, /profiles/{id}, .../religious-preference, .../marriage-intention, .../partner-preference, .../addresses, .../education	Core CRUD plus 1:1 and 1:N sub-resources
Wali	/profiles/{id}/Wali, /conversations/{id}/Wali-approval, /conversations/{id}/share-wali-contact	Server-side authority checks are the highest-risk logic in the whole API
WaliChangeRequest	/Wali-change-requests, /Wali-change-requests/{id}/decision	Reviewer identity branches by RequestType (candidate vs Admin)
Messaging	GET/POST /conversations/{id}/messages	REST write path, sole place approval is enforced; delivery fan-out via Supabase Realtime (Section 8), not this endpoint
Subscription	/subscriptions/checkout, /payments/webhook/razorpay, /payments/webhook/stripe, /subscriptions/current	Gateway and final amount always computed server-side
Matching	GET /matches	Never exposes HIGH-classified fields (e.g. Wali contact)
Media	/profiles/{id}/photos, /profiles/{id}/video	Max 4 photos, 1 video ≤ 60s, both moderation-gated, stored in Supabase Storage
Verification	/profiles/{id}/verification-requests	
Moderation	/reports, /admin/reports	Exactly one of reportedProfileId/reportedMessageId
Admin	/admin/users/{id}/role	Role change restricted to Role=Admin

Realtime channel (not in the REST spec): Android subscribes directly to **Supabase Realtime** (not a custom WebSocket service) to receive message push and Wali-approval status changes. Subscription access is gated by a Row-Level Security policy checking ConversationApproval/*ManagedByWaliId at the database level — a second, independent check underneath the REST endpoint's own application-level enforcement, not a replacement for it.

12. Security Architecture

Area	Control
Regulatory scope	DPDP Act 2023 (India) is the primary target for Phase 1: lawful basis for processing sensitive personal data (religion, health/disability, family details), data minimization (Fields Never Collected list), and a defined data-retention stance to be set before launch.
Authentication (v1.2)	Supabase Auth (self-hosted GoTrue), phone-OTP. Every request to the Next.js API verifies the Supabase-issued JWT server-side — the API never trusts a client-supplied identity claim, only a cryptographically verified token.
Wali authority	Every authority check (approve/reject, contact disclosure, messaging routing) is enforced server-side, in Next.js, against Wali.ControlLevel/IsActive — never inferred from a client-supplied flag, which screen the request came from, or a database policy alone (confirmed decision, ADR-009).
No ID verification for Walis	Accepted MVP risk (confirmed decision) — a Wali's identity is not verified beyond the candidate's own attestation. Documented as ADR-007; revisit if abuse patterns emerge.
Encryption	Email/Phone storage, hashing, and OTP secrets are now Supabase Auth's responsibility entirely, not this schema's (v1.2) — the previous MobileNumber-encryption open question is resolved by removing that responsibility from our own tables. Photo/Video access-controlled via signed URLs (Supabase Storage), not encrypted at rest.
Payment security	Razorpay/Stripe webhook signatures verified server-side on every call (X-Razorpay-Signature / Stripe-Signature); final charge amount always computed server-side, never trusted from the client.
Media	Signed-URL-only access for all photos/video via Supabase Storage; moderation gate enforced before any visibility, at the API/serialization layer so a hidden field can never leak in a raw response.
Rate limiting	OTP request/verify is now Supabase Auth's own rate-limiting responsibility. Contact-disclosure actions remain capped at one per side per conversation, enforced by our own API.
Realtime channel security (v1.2)	Supabase Realtime subscriptions are gated by a Row-Level Security policy checking ConversationApproval/*ManagedByWalid at the database level — a second, independent layer underneath the Next.js API's own enforcement. A revoked approval stops new rows reaching a subscriber automatically, without a separate "kick the connection" step.

13. Infrastructure & Deployment

Phase 1: self-hosted VPS (India region), Docker Compose, including Supabase's official self-hosted stack (v1.2) — ADR-005's self-hosting principle is preserved, not reversed, by the Supabase migration. See the deployment diagram on the following landscape page.

Nikah Platform — Deployment Diagram (Self-Hosted VPS with Supabase, v1.2)

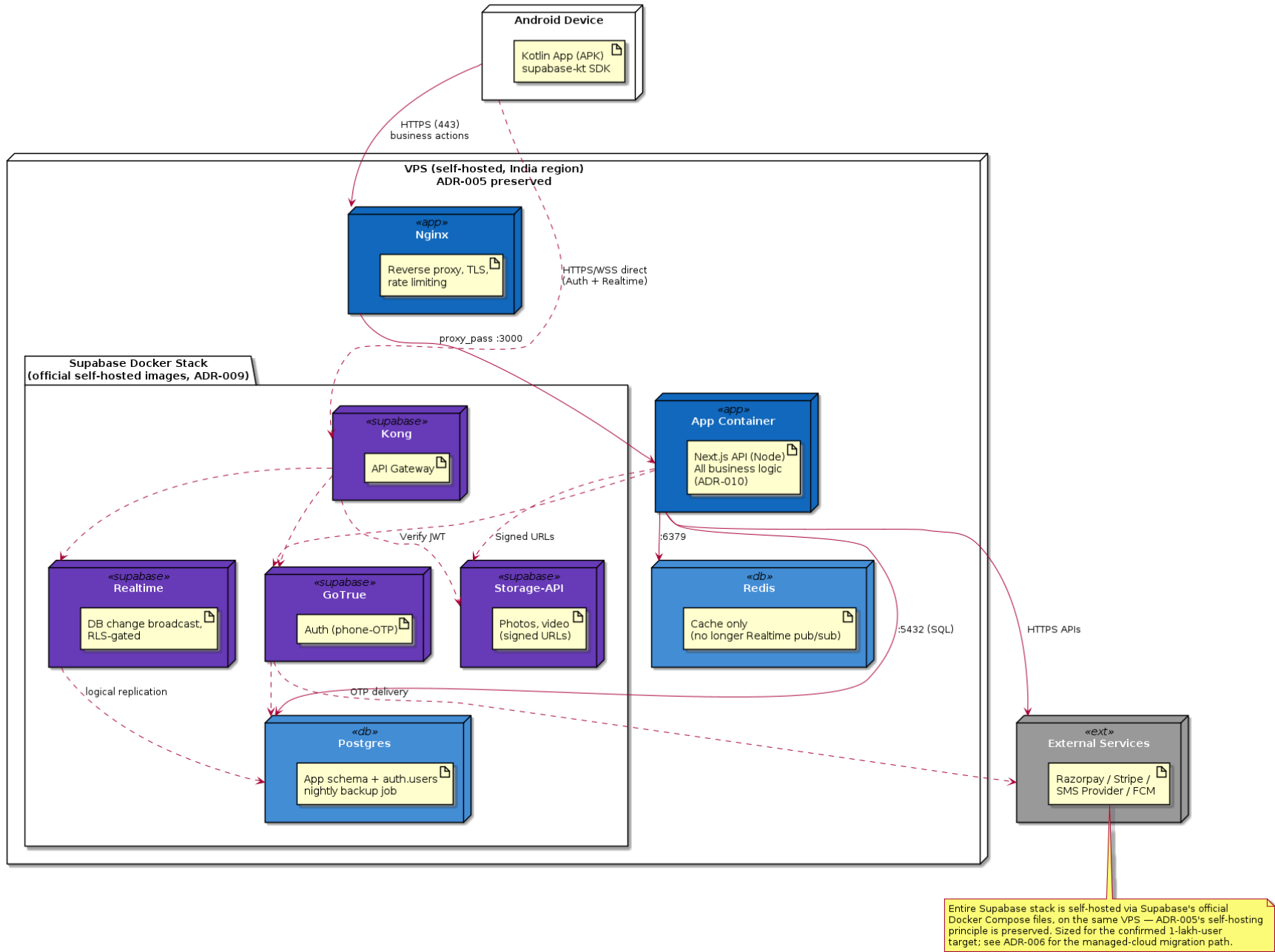


Figure 6 — Deployment Diagram (PlantUML, Self-Hosted VPS with Supabase, v1.2)

13.1 Notes

- **Containers:** Nginx (reverse proxy/TLS), App (Next.js REST API), Redis (cache only, no longer Realtime pub/sub), and Supabase's own stack (Kong gateway, GoTrue/Auth, Realtime, Storage-API, Postgres) — all as one Docker Compose deployment on one VPS for Phase 1.
- **What Supabase's self-hosted stack adds operationally:** more containers to monitor than the v1.1 design, but each is officially maintained by Supabase rather than hand-rolled (no more custom Socket.IO service to patch and secure ourselves).
- **Backups:** nightly Postgres dump (the same database Supabase's Auth/Storage/Realtime components share with our app schema) to off-VPS storage, retained on a rolling window; restore procedure tested before launch, not just assumed to work.
- **TLS:** terminated at Nginx; Let's Encrypt auto-renewal.
- **Scaling path (see ADR-006):** if the 1-lakh target is exceeded, Postgres is the first candidate to move to a managed service (Supabase's own hosted cloud is one option, since the schema is already Supabase-shaped) without changing any application container boundary above.
- **Mobile release channel:** Android app distributed via Google Play (staged rollout); backend API versioned (`/v1`) so app updates and backend deploys are decoupled. The app talks to Kong (Supabase's gateway) directly for Auth/Realtime, and to the Next.js API directly for everything else.

14. Architecture Decisions (ADRs)

ADR-001: Modular Monolith over Microservices [Accepted]

Context	Solo-developer, AI-assisted (Claude Code) build; small team, moderate scale target.
Decision	Single Next.js deployment with enforced module boundaries (folder structure, no cross-module direct table access), all on one PostgreSQL database.
Consequences	Faster to build and reason about now; each module can be extracted into its own service later with minimal rework, since boundaries are already code-enforced.

ADR-002: Native Kotlin Android App First, Web Second [Accepted]

Context	Founder's explicit priority: mobile app first, full website afterward.
Decision	Android app in Kotlin (Jetpack Compose) as the primary client, consuming the same REST/WebSocket API a future web app will use. iOS deferred.
Consequences	Backend API must stay client-agnostic from day one, since two clients (Android, then Web) consume it. No iOS at launch is an accepted scope reduction (Section 16).

ADR-003: Dedicated Realtime Messaging Service for Instant Delivery [Superseded by ADR-009]

Context	Instant, WhatsApp-like message delivery is a hard requirement, not best-effort.
Decision	Originally: a custom WebSocket server (Socket.IO) backed by Redis pub/sub, alongside the REST message-send endpoint. v1.2: replaced by Supabase Realtime (ADR-009) — same principle (instant push, REST remains the authoritative write path), different implementation.
Consequences	Superseded rather than deleted from this record, since the underlying requirement and the “REST stays authoritative” principle carried forward unchanged into ADR-009.

ADR-004: Dual Payment Gateway: Razorpay (India) / Stripe (International) [Accepted]

Context	India-first launch, but the schema already supports 10+ countries.
Decision	Server-side gateway selection by Profile.Country; final amount always computed server-side.
Consequences	Two webhook integrations to maintain, each with its own signature verification — kept as separate endpoints rather than a forced-generic one, for security clarity.

ADR-005: Self-Hosted VPS over Managed Cloud (Phase 1) [Accepted]

Context	Founder preference; target scale (up to 1 lakh users) doesn't require managed-cloud elasticity.
Decision	Single VPS, Docker Compose, India region. v1.2: still holds after adopting Supabase (ADR-009), since Supabase's official stack is self-hostable via Docker — adopting Supabase did not require reversing this decision.
Consequences	Lower cost, full control; founder takes on more ops responsibility (backups, patching, uptime monitoring, and now also Supabase's own containers) than a managed platform would require.

ADR-006: Defined Migration Path to Managed Cloud [Proposed]

Context	Scale could exceed the VPS's capacity if growth outpaces the 1-lakh estimate.
Decision	Postgres is the primary candidate to move to a managed service without changing any application container boundary. v1.2: Supabase's own hosted cloud is now a natural migration target, since the schema and Auth/Storage/Realtime setup are already Supabase-shaped — migrating later is a hosting change, not a re-architecture.
Consequences	No work required now; this ADR exists so the migration isn't a scramble if/when it's needed.

ADR-007: No ID Verification Required for Walis (Wali) in MVP [Accepted]

Context	Founder decision, balancing user friction against fraud risk for launch.
Decision	A Wali's identity is accepted on the candidate's own attestation (name + relation); VerificationRequest applies only to the candidate profile, not the Wali.
Consequences	Real, accepted risk: a person could falsely claim to be someone's Wali. Revisit if abuse patterns are observed post-launch (see Section 16).

ADR-008: Manual Testing Now, Automated Testing Later [Accepted]

Context	Founder resourcing at this stage of the project.
Decision	Manual QA checklist for Phase 1 releases; automated test suite (Section 15) built out once the core flows stabilize.
Consequences	Faster initial iteration; real risk of regressions creeping in until automation lands — Section 15 defines the trigger point for when to invest in it.

ADR-009: Adopt Self-Hosted Supabase for Auth, Storage, and Realtime [Accepted]

Context	Founder decision to reduce custom code for identity, file storage, and instant messaging, while keeping the self-hosting principle (ADR-005) intact and gaining an official Kotlin SDK for Android.
Decision	Self-host Supabase's open-source Docker stack (GoTrue/Auth, Storage-API, Realtime, Postgres) on the same VPS. Supabase Auth's phone-OTP replaces the custom User/Session/OTP entities entirely. Supabase Realtime replaces the custom Socket.IO service (supersedes ADR-003). Supabase Storage replaces generic S3-compatible storage (a near-zero-cost swap, since Supabase Storage is itself S3-compatible).
Consequences	Fewer custom auth/realtime/storage code paths to build and secure ourselves; genuine security upside from Row-Level-Security-gated Realtime subscriptions (Section 8). Real cost: more containers to operate (Kong, GoTrue, Storage-API, Realtime alongside Postgres), and a new dependency on Supabase's own upgrade cadence and documentation quality.

ADR-010: Business Logic Stays in Next.js, Not Supabase RLS/Edge Functions

[Accepted]

Context	Supabase offers Edge Functions and an auto-generated PostgREST API that could host application logic directly; adopting Supabase for infrastructure raised the question of whether to also adopt its application-layer paradigm.
----------------	--

Decision	All authorization and business-rule logic (Wali-approval branching, contact disclosure, gateway selection, discount calculation) stays exclusively in the Next.js API layer. Supabase Row-Level Security is used only as a defense-in-depth layer for Realtime/Storage access, never as the primary place a rule is decided.
Consequences	One codebase, one language, one place to read the whole story of a complex decision — easier to review and test than logic split across SQL policies and small serverless functions. Trade-off: doesn't take advantage of Supabase's auto-generated API or Edge Function convenience for simpler CRUD, by choice, for consistency.

15. Testing Strategy

Now (manual): a structured manual QA checklist run before every release, covering the highest-risk flows first — Wali approve/reject, contact disclosure, messaging block/unblock, payment checkout for both gateways, and the completion-percentage search gate.

Automation roadmap (trigger: once core flows stabilize, per ADR-008):

Layer	Tooling	Covers
Android unit/instrumentation	JUnit, Espresso	Kotlin business logic, UI flows on-device
Backend unit/integration	Vitest or Jest + Supertest	API route handlers, especially Wali-authority branching logic
End-to-end API	Postman/Newman or Playwright API testing	Full request/response contracts against the OpenAPI spec
Load/performance	k6 or Artillery	Validate the 1-lakh-scale concurrent-user NFR target before launch, not after
Security	OWASP ZAP baseline scan	Webhook signature bypass attempts, session/auth edge cases

Highest-priority test cases (business-rule risk, not just code coverage): a display-only Wali must never be able to approve/reject; a revoked Wali approval must block new messages immediately without deleting history; a Swap must clear any in-flight ManagedByWalild rather than silently transferring it; webhook signature verification must reject a forged payload from either gateway.

16. Future Enhancements

- **iOS app** — deferred; Android (Kotlin) ships first.
- **Full website** (Phase 2) — consumer-facing, not just the Admin Portal.
- **Voice calling** — in scope for later (confirmed); video calling explicitly out of scope. Likely needs a third-party voice SDK (e.g. Agora, Twilio Voice) rather than building WebRTC signaling in-house. Design question resolved: **an active, Proxy-control Wali may join a voice call**, mirroring his messaging authority — when voice calling is designed, the call session should carry the same per-side ManagedByWalild concept already used for messaging, so a Wali who controls a side's conversation can join or monitor its calls too, not just its text. Full call-signaling and consent-notice design (e.g. does the other party see "Wali joined the call") is deferred to when this feature is actually scheduled, not designed here.
- **Cloud migration** if VPS capacity is exceeded (ADR-006).
- **Country-aware Income bands and Address regions** — currently India-Rupee-specific/India-structured; needs localization before a genuine non-India launch.
- **MobileNumber encryption** — resolved (v1.2): now Supabase Auth's responsibility, not this schema's. No longer an open item.
- **Quick-Match Wizard refinement** — reconciling its ephemeral answers more richly into PartnerPreference once registration completes.
- **Fine-grained Admin permissions** — currently Role=Admin is all-or-nothing; may need can-verify/can-manage-subscriptions/can-view-reports separation as the team grows beyond a solo founder.
- **Wali ID verification** — revisit ADR-007 if abuse patterns emerge post-launch.

Appendix B — Open Issues

Carried forward from the Architecture Design Document v3.4 (Appendix A has the full table with rationale):

#	Issue	Status
1	Income bands remain India-Rupee-specific despite multi-country schema support	Known limitation, non-blocking for MVP

Appendix C — Glossary

Term	Meaning
Wali	Islamic marriage Wali — a relative (father, mother, uncle, brother, etc.) with a recognized role in a woman's marriage process.
Nikah	The Islamic marriage contract/ceremony.
Madhhab	A school of Islamic jurisprudence (e.g. Hanafi, Maliki, Shafi'i, Hanbali).
Sect	Broad Islamic denomination (e.g. Sunni, Shia).
Mahr	A mandatory gift from the groom to the bride as part of the marriage contract.
Nafaqah	The husband's financial maintenance obligation to his wife under Islamic law.
Revert	A person who has converted to Islam.
OTP	One-Time Password — a short-lived code used for phone/email verification and login.
ADR	Architecture Decision Record — a short document capturing one architectural decision, its context, and its consequences.
C4 Model	A standard notation for architecture diagrams at four zoom levels: Context, Container, Component, Code.
Proxy (Wali ControlLevel)	A Wali with full decision-making authority over a profile.
DisplayOnly (Wali TrustLevel/ControlLevel)	A Wali listed for notification purposes only, with no authority.

Appendix D — Reference Documents

Document	Contents
Architecture Design Document v3.4	Full Entity Catalog (32 entities), Attribute Classification, Relationship Matrix, ER Diagram, complete revision history — reproduced in full as Appendix A.
Nikah_Platform_OpenAPI_v0.5.yaml	Machine-readable OpenAPI 3.0 specification (validated), 24 endpoints, ready for Claude Code to scaffold against directly.
react-app-beta-ruby.vercel.app	Live front-end prototype, audited twice against this design; source of the Quick-Match Wizard, video-introduction, and disability-type-list confirmations.
FORM_SPEC.md / FORM_FUNCTIONALITY.md	Original field-level requirements (Sprint 1 source of truth).

Appendix A

Full Entity Catalog, Attribute Classification, Relationship Matrix & ER Diagram

Reproduced in full from the companion Architecture Design Document v3.4.
The following pages are that document in its entirety — System Context, Business Modules,
the complete 32-entity catalog with business rules and security classification, the full Attribute
Classification and Relationship Matrix tables, the ER diagram, cross-cutting business rules,
and its own revision history and open issues.

Nikah Platform

Sharia-Compliant Muslim Matrimony Platform Software Architecture Design Document

Document version	3.4 (Sprint 1 + Sprint 3, backend migrated to self-hosted Supabase)
Source materials	FORM_SPEC.md v1.4, Final_Architecture_design.docx (Sprints 1 & 3 draft), UMLDiagrams
Architecture style	Modular Monolith (Next.js + PostgreSQL), microservice-ready module boundaries
Reviewed by	Architecture mentorship session

This document supersedes the draft *Final_Architecture_design.docx*. Every entity, attribute, relationship and business rule below was walked through and confirmed in a structured architecture review; corrections and rationale for each change are called out inline, and unresolved items are isolated in the Open Issues appendix rather than silently decided.

Table of Contents

1. Revision Summary — What Changed From the Draft
2. System Context (Actors & External Systems)
3. Business Modules
4. Entity Catalog
5. Attribute Classification (Security & Privacy)
6. Relationship Matrix
7. Entity-Relationship Diagram
8. Cross-Cutting Business Rules
9. Open Issues & Deferred Scope

1. Revision Summary — What Changed From the Draft

The original draft (*Final_Architecture_design.docx*) and its accompanying ER diagram disagreed with each other in several places, and both were incomplete relative to the field-level requirements captured in *FORM_SPEC.md* during Sprint 1. The table below is the audit trail of every correction made and the decision that resolved it.

#	Issue found in draft	Resolution
1	Entity #9 was headed "MatchCandidte" but its attributes/relationships were actually the Wali entity. MatchCandidate had no real definition anywhere.	Wali restored as its own entity. MatchCandidate rebuilt from the ER diagram fields, extended with CompatibilityScore, TrustScoreAtMatch, BlendedScore, and MatchedProfileVerified.
2	PartnerPreference primary key typo: "preferpartanerId".	Renamed to PreferenceId, consistent with the rest of the schema's naming convention.
3	Admin and Report referenced in the Relationship Matrix but never defined anywhere, including the ER diagram.	No separate Admin entity created. Added Role (Candidate / Admin / Moderator) to User. Report defined as its own entity, targeting either a Profile or a Message.
4	Nine entities visible in the ER diagram (Session, OTP, LoginLog, SubscriptionPlan, Education, Occupation, Address, PrivacySetting, Attachment) had no Entity Catalog record, no business rules, no security classification.	All nine fully catalogued below with attributes, business rules and security classification.
5	Profile modeled only ~19 generic fields; none of the ~90 fields catalogued from FORM_SPEC.md in Sprint 1 (sub-sect, madhhab, hijab/beard, disability, children detail, Wali display fields, religiosity, hobbies, wedding preferences, Nafaqah/Mahr, etc.) were represented.	Profile split into six focused entities (Profile core, ReligiousPreference, MarriageIntention, ProfileChildrenDetail, plus multi-value child tables) so the schema traces cleanly back to every field in the approved form spec.
6	Two distinct "Wali" concepts — (a) someone registering on a candidate's behalf, (b) a candidate simply listing a relative's contact details for display — were conflated into one Wali record requiring a login.	Wali.UserId made nullable. Added TrustLevel (Registered vs. DisplayOnly) so a display-only Wali with no account is explicitly marked lower-trust than a Wali who has actually registered and verified.
7	Conversation carried a flat WaliApprovalStatus column with no way to represent a Wali revoking, then later re-approving, a conversation — and no way to know if one or both families' Walis must sign off.	New WaliApproval entity: one append-only row per approve/revoke decision, tagged with Side (Initiator/Receiver). Both sides' Walis gate independently — a conversation needs both green. Conversation keeps a denormalized CurrentApprovalStatus for fast checks.
8	MatchCandidate.Score conflated compatibility matching with trust/verification in an undocumented way.	CompatibilityScore (pure PartnerPreference match) and TrustScoreAtMatch are stored separately, then combined into BlendedScore. Verification-status changes update the existing row rather than requiring a new match to be generated; unverified profiles still surface in results, just flagged MatchedProfileVerified = false.
9	Height stored as two separately-updatable fields (feet and cm) in the draft's implied schema, risking the two falling out of sync.	Single canonical HeightCm column; feet/inches are a display-layer conversion only, never persisted.
10	Message carried its own ApprovalStatus column duplicating Conversation-level Wali approval, with no distinction from AI content moderation.	Message.ApprovalStatus removed. Wali approval lives only at the Conversation/WaliApproval level. A separate ModerationFlag on Message covers automated content moderation, a genuinely different concern.

#	Issue found in draft	Resolution
11	Draft listed Parent as a fourth primary-user actor, separate from Wali, with overlapping responsibilities.	Parent removed as a separate actor. A parent is simply one possible value of Wali.Relation (alongside uncle, brother, etc.) — no separate entity or actor needed; Wali's interaction list now folds in “participate in communication.”
12	Open Issue: Address entity's purpose overlapped ambiguously with Profile's own Country/State/City fields.	Resolved: Address now explicitly holds exactly two typed rows per profile — AddressType = Native (primary) and Current (secondary); the two may hold identical values if the person hasn't relocated. Profile.NativePlace (a duplicate free-text field) is removed; Profile.Country/State/City is retained only as a denormalized copy of the Current address, kept for fast search filtering without a join.
13	Open Issue: Admin permission granularity undecided.	Resolved for v1: Role = Admin grants full administrative permission across all modules. No separate fine-grained permission table in this version; revisit only if a real need for partial-admin roles emerges.
14	Open Issue: Photo.StorageKey encryption-at-rest undecided.	Resolved: no at-rest encryption required. Access control is enforced entirely through signed URLs and the Media module's moderation gate.
15	Major requirements change: the Wali model assumed a single “trust level” and a single dual-Wali approval mechanism, with no distinction between a Wali who registers by proxy (full control) and one who is merely listed for display (no control).	Wali.TrustLevel split into two independent fields: VerificationLevel (is this really them) and ControlLevel (can they act for the candidate). A Wali can verify their identity without ever gaining control, and vice versa — these no longer have to move together.
16	No mechanism existed for a display-only Wali to be promoted to full control, or for a candidate to swap out her Wali for someone else — both are now required, and are approved by two different parties (the candidate vs. Admin).	New WaliChangeRequest entity with RequestType ∈ {Promotion, Swap}. Promotion is reviewed by the candidate herself; Swap requires a mandatory reason and is reviewed by Admin. One entity, two reviewer rules, instead of duplicated logic.
17	WaliApproval assumed every conversation side always has a Wali; it had no way to represent a candidate approving on her own behalf when no Wali is active, and no way to represent a Wali actually taking over the messaging itself (as opposed to just approving that it can happen).	Renamed to ConversationApproval; ApprovingUserId now uniformly represents whoever currently controls that side (Wali or candidate). Conversation gained per-side *ManagedByWaliId fields, so once a Wali approves, all further messages on that side must come from the Wali's account — the candidate is excluded from that specific conversation. Message gained SentByUserId, distinct from SenderProfileId, so the system always knows who actually typed a message even though the other party only sees one counterparty.
18	Contact-info sharing with a match was only an implied static PrivacySetting flag, with no way to represent whose number gets shared (Wali's vs. candidate's), why, or as an auditable event — including the fallback case where a candidate manually shares her Wali's number because his login is unavailable.	New ContactDisclosure entity: one audited row per side per conversation, recording DisclosedContactType (WaliContact/CandidateContact), what triggered it (Wali approval, manual share by the candidate, or no Wali at all), and who triggered it.
19	Open Issue: whether email/mobile encryption was needed now, and if so how to keep it queryable.	Resolved: not encrypted for MVP. MobileNumber will be encrypted in a later production phase (mechanism deferred, must stay queryable); Email remains unencrypted permanently.

#	Issue found in draft	Resolution
20	Open Issue: what happens to in-app messaging when a side's Wali's login is unavailable and the candidate manually shares his contact instead.	Resolved: the conversation itself stays fully Blocked — no fallback to the candidate messaging directly. The only thing allowed through is one narrow exception: a single system-generated message (Message.MessageType = ContactShareNotice) carrying the Wali's contact info plus the candidate's own free-text remark. ContactDisclosure gained a Remark field and a MessageId link to record exactly this.
21	Open Issue: what PartnerPreference.WaliInvolvementExpected actually means given the new Wali control model.	Resolved at the time as a hard Matching filter (row 21) — superseded by row 23 below: a live UI audit confirmed the feature isn't required at all and it has been removed entirely.
22	A live prototype UI audit (react-app-beta-ruby.vercel.app) was performed to validate the design against a real, built interface — registration, Quick-Match Wizard, and all 4 parts of the profile form.	Confirmed correct as designed: Education as a repeatable list (live prototype's single dropdown will be updated to match), ProfileDisabilityType's multi-select-on-Yes shape (with the validated type list now captured), the phone-privacy copy (refers to User.MobileNumber, does not contradict ContactDisclosure), and Wali.Relation as free text for any relative (live prototype's son/daughter-only registration will be updated to match).
23	PartnerPreference.WaliInvolvementExpected and ReligiousPreference.ProposalContactPref — both confirmed, during the live UI audit, as not required for this platform.	Both removed entirely. WaliInvolvementExpected's hard-filter Matching rule (row 21) is void. There is no candidate-selectable messaging-routing preference — routing remains governed solely by Wali.ControlLevel/active status, exactly as already designed.
24	Live UI audit surfaced a genuinely new feature not previously modeled: an optional short video introduction (candidate- or Wali-recorded).	New ProfileVideo entity added, mirroring Photo's shape (signed-URL access, moderation gate), plus the 4-photo maximum and Profile.RoleModel ("someone you look up to") confirmed/added from the same audit.
25	Quick-Match Wizard (Open Issue, formerly #2) confirmed live and fully built during the UI audit — an 8-step bilingual (English/Hindi) pre-registration teaser flow.	New QuickMatchPreference entity added, capturing its exact shape; resolves the open issue definitively rather than leaving it pending.
26	Mandatory-field list for ReligiousPreference didn't match the live prototype (which requires Salah frequency and Hijab/Beard, not just Smoking/Alcohol).	ReligiousPreference's business rules updated to state the validated mandatory set: Smoking, Alcohol, SalahFrequency, and the applicable gender-conditional field.
27	Second live UI audit pass (react-app-beta-ruby.vercel.app), covering the self-registration path (previously only the Wali-proxy path had been walked end to end).	Confirmed: ProposalContactPref's removal (row 23) is now also reflected in the live UI — replaced with a free-text Wali Name + a Relation quick-pick (Father/Brother/Uncle/Appointed Wali). Wali.Phone made explicitly nullable: the live UI never asks for a display-only Wali's phone number, only Name and Relation. The live UI's Part 2 tri-state Wali-involvement field and Part 1's duplicate "who is filling this" question are both still present unchanged — expected, since their removal/simplification (rows 21, 23, and the UI-alignment guidance rule) is a future-implementation decision, not yet reflected in this front-end-only prototype.

#	Issue found in draft	Resolution
28	Location fields simplified in the live UI: Part 1 now collects only Country + a free-text "native place/hometown" string — the separate State dropdown (previously flagged as India-only) has been removed entirely, not fixed.	Resolves half of the former Open Issue #2 (State) by elimination. Income remains India-Rupee-specific (new Open Issue #2). Whether Address should simplify to match (Country + free text only) or keep its richer structured City/State shape is now Open Issue #3, pending your decision.
29	Open Issue: Address structure — keep City/State as structured fields, or simplify to match the live UI's Country + free-text approach?	Resolved: keep City/State as separate structured fields. No schema change — confirms the design already in place.
30	Major infrastructure decision: backend migrated to Supabase (self-hosted, keeping ADR-005's self-hosting principle intact) for Auth, Storage, and Realtime. Business logic (Wali-approval branching, contact disclosure, gateway selection, discount calculation) deliberately stays in the Next.js application layer, not in database RLS policies or Edge Functions — that logic is too conditionally complex to reason about split across two systems.	User entity reduced to a thin Role/Status extension of Supabase's own auth.users; Session and OTP entities removed entirely — Supabase Auth (phone-OTP) now owns identity, credentials, and session tokens outright. LoginLog retained, now populated by a trigger reacting to Supabase Auth events rather than our own login code. The previously-open MobileNumber-encryption question is moot — that responsibility now belongs to Supabase Auth, not this schema.

2. System Context

2.1 Actors

Actor	Type	Key Interactions
Bride / Groom	Primary user	Register, complete profile, save draft, upload photos, search/view matches, express interest, manage privacy, receive notifications
Wali	Primary user	Two distinct modes: (1) Proxy — registers the profile directly (or is later promoted with the candidate's approval), gets a full independent login, and has complete decision-making authority over that profile, including approving/rejecting conversations and revealing contact info; (2) Display-only — just listed by a self-registered candidate for notification purposes (new match, conversation started, marriage-stage progress), with zero approval authority. Exactly one Wali slot per profile; can be swapped only via Admin-approved request. One Proxy Wali account can manage multiple candidates' profiles (e.g. a father managing more than one daughter).
Admin	Internal user (User.Role)	Verify profiles, manage users & subscription plans, review reports, audit system activity
Moderator	Internal user (User.Role)	Review reported profiles, moderate messages, escalate serious cases to Admin
Payment Gateway	External system	Process payments, return payment status
SMS/Email Provider	External system	Send OTP, email & SMS notifications
Identity Verification Provider (future)	External system	Verify identity documents, return verification status
Object Storage (S3-compatible)	External system	Persist photos and message attachments; access only via signed URLs

3. Business Modules

Module boundaries are drawn so each can later be extracted into its own microservice with minimal rework — each owns a clear attribute set and a clear set of entities (Sprint 2 territory in full, summarized here for traceability).

Module	Responsibilities	Owned Entities
Authentication	Identity, phone-OTP login, and session management — all delegated to Supabase Auth (self-hosted); this module now only extends it with app-specific Role/Status and audit logging	User (thin extension), LoginLog
User Profile	Create/edit profile, religious details, marriage intentions, background details, profile completion scoring	Profile, ReligiousPreference, MarriageIntention, ProfileChildrenDetail, ProfileDisabilityType, ProfileLanguage, ProfileHobby, Education, Occupation, Address
Partner Preference	Capture and expose what a candidate is looking for in a partner	PartnerPreference
Wali	Wali registration (proxy or display-only), promotion/swap requests, per-conversation approval and contact disclosure	Wali, WaliChangeRequest, ContactDisclosure, ConversationApproval
Verification	Phone/identity/photo verification, trust scoring	VerificationRequest
Matching	Generate candidates, blend compatibility with trust, ranking	MatchCandidate
Search	Search profiles, filter, sort, paginate (reads Profile + PartnerPreference)	— (query-only over Profile/PartnerPreference)
Messaging	Conversations, messages, attachments, Wali approval gate, moderation flag	Conversation, Message, Attachment
Subscription	Plans, payments, renewals, premium access, revert/divorce/age/widowed discount rules	SubscriptionPlan, Subscription, Payment
Media	Upload/delete photos, moderation, signed URL generation	Photo
Privacy	Photo/contact/profile visibility rules	PrivacySetting
Notification	SMS, email, push, match & verification alerts	— (event-driven, no owned tables in v1)
Reporting/Moderation	User reports against profiles or messages, resolution tracking	Report
Administration & Audit	User/role/subscription management via User.Role; login/activity logs	(reads across modules) + LoginLog

4. Entity Catalog

Format: Purpose → Key Attributes → Primary/Foreign Keys → Business Rules → Security Classification. Entities are grouped by owning module.

Entity	User
Purpose	App-specific extension of a Supabase Auth identity — holds only what Supabase's own auth.users table doesn't: role and app-level status. Superseded design (v1.2): Email, MobileNumber, PasswordHash, OTP handling, and session tokens are no longer modeled here at all — Supabase Auth (phone-OTP) owns all of that.
Key Attributes	UserId (= Supabase auth.users.id), Role, Status, CreatedAt, UpdatedAt
Primary Key	UserId
Foreign Keys	UserId → Supabase auth.users.id (one-to-one, not a table we own)
Business Rules	Role ∈ {Candidate, Admin, Moderator}; account cannot access app features unless Status = Active. This row is created by a database trigger the moment Supabase Auth creates a new auth.users row — the application never creates a User row directly.
Security Class.	MEDIUM
Change from draft	<i>v1.2: reduced from a full custom identity table to a thin Role/Status extension. Email/MobileNumber/PasswordHash/verification flags removed — now Supabase Auth's responsibility, not ours to store or validate.</i>

Entity	LoginLog
Purpose	Audit trail of login attempts, for security review and the Administration/Audit module.
Key Attributes	LoginLogId, UserId (FK), LoginTime, IpAddress, LoginStatus
Primary Key	LoginLogId
Foreign Keys	UserId → User
Business Rules	Append-only; never updated or deleted by application code. Populated by a database trigger/webhook reacting to Supabase Auth login events — not written directly by our own login code, since Supabase Auth now owns the login mechanism itself.
Security Class.	MEDIUM
Change from draft	<i>v1.2: population mechanism changed from “our own OTP-verify endpoint writes this row” to “a trigger reacts to a Supabase Auth event” — the table's shape and purpose are unchanged.</i>

Entity	Profile
Purpose	Core, searchable matrimonial profile — trimmed to fields needed for search/matching; detailed sections split into dedicated entities below.
Key Attributes	ProfileId, UserId (FK), FirstName, LastName, Gender, DateOfBirth, MaritalStatus, Sect, SubSect, IsRevert, Education, Income, Country, State, City, HeightCm, Bio, RoleModel, VisibilityStatus, CompletionPercentage, CreatedAt, UpdatedAt
Primary Key	ProfileId
Foreign Keys	UserId → User (1:1)

Business Rules	One User owns one Profile. Must reach CompletionPercentage ≥ 60% before appearing in search. Female profiles default to private visibility. IsRevert feeds the men's subscription revert-discount rule; DateOfBirth and MaritalStatus feed the age/divorced/widowed discount rules (highest single discount only, no stacking). Country/State/City are a denormalized copy of the Current-type Address row, refreshed whenever that Address changes — kept here only so Search can filter by location without a join; Address is the source of truth.
Security Class.	HIGH
Change from draft	<i>Trimmed from the draft's 19 generic fields; SubSect and IsRevert added (both required by FORM_SPEC.md but missing from the draft); religious/lifestyle/marriage-intention/background fields moved to their own entities below. NativePlace removed — superseded by Address(AddressType=Native). RoleModel ("someone you look up to," free text, optional) added — discovered during the live UI audit.</i>

Entity	ReligiousPreference
Purpose	All of a candidate's own religious-practice and lifestyle answers (Part 3 of the profile form) — kept separate from core Profile since it's rarely queried for search filtering and changes on a different cadence.
Key Attributes	ReligiousPreferenceId, ProfileId (FK, 1:1), Madhhab, SalahFrequency, ReligiosityLevel, HijabStatus, BeardStatus, Hifz, QuranReadingHabit, IslamicEducationLevel, StudyIslamInterest, HalalDiet, Fasting, Zakat, Smoking, Alcohol, HalalEntertainment, MixedGenderComfort, MusicViews, Nafaqah, WifeWorkingAttitude, MahramWali, HousingStatus, MahrCapacity, CareerAfterMarriage, MahrExpectation, WeddingTypePreference, IslamicParentingStyle, RoleVision, DeenNotes
Primary Key	ReligiousPreferenceId
Foreign Keys	ProfileId → Profile
Business Rules	HijabStatus shown, and mandatory, only if Gender=Female; BeardStatus shown, and mandatory, only if Male. Nafaqah/WifeWorkingAttitude/MahramWali/HousingStatus/MahrCapacity shown only for Male profiles; CareerAfterMarriage/MahrExpectation shown only for Female profiles. Mandatory fields (validated against the live prototype): Smoking, Alcohol, SalahFrequency, and the applicable gender-conditional field (HijabStatus or BeardStatus). Everything else in this entity is optional.
Security Class.	MEDIUM
Change from draft	<i>Removed ProposalContactPref entirely — confirmed not required for this platform; candidates do not choose a proposal-routing preference, so there is no hybrid "direct + Wali CC" messaging mode to design for. Messaging routing remains governed solely by Wali.ControlLevel/active status via Conversation.*ManagedByWalid, as already designed. Mandatory-field list corrected to match the live prototype (Salah and Hijab/Beard are required, not merely Smoking/Alcohol as previously documented).</i>

Entity	ProfileLifestyleGoal
Purpose	Multi-select Islamic lifestyle goals.
Key Attributes	Id, ReligiousPreferenceId (FK), Goal
Primary Key	Id
Foreign Keys	ReligiousPreferenceId → ReligiousPreference
Business Rules	One row per selected goal.
Security Class.	LOW
Change from draft	<i>New — multi-select fields cannot be flat columns.</i>

Entity	ProfileWeddingRitual
Purpose	Multi-select wedding rituals/functions.
Key Attributes	Id, ReligiousPreferenceId (FK), Ritual
Primary Key	Id
Foreign Keys	ReligiousPreferenceId → ReligiousPreference
Business Rules	Populated only when WeddingTypePreference = Large wedding.
Security Class.	LOW
Change from draft	<i>New.</i>

Entity	MarriageIntention
Purpose	Timeline and household-formation intentions (Part 1, Marriage Intentions).
Key Attributes	MarriageIntentionId, ProfileId (FK, 1:1), WhenToMarry, OpenToRelocation, WantChildren, FamilyStructure, WhoIsSearching
Primary Key	MarriageIntentionId
Foreign Keys	ProfileId → Profile
Business Rules	WhenToMarry and WhoIsSearching are mandatory; the rest optional.
Security Class.	LOW
Change from draft	<i>New.</i>

Entity	ProfileChildrenDetail
Purpose	Children-related detail, only relevant to divorced/widowed candidates.
Key Attributes	Id, ProfileId (FK, 1:1 conditional), DivorceCertificate, ChildrenCount, ChildrenCustody, ChildrenDetailsText
Primary Key	Id
Foreign Keys	ProfileId → Profile
Business Rules	Row exists only if MaritalStatus ∈ {Divorced, Widowed}. DivorceCertificate required if MaritalStatus = Divorced.
Security Class.	HIGH
Change from draft	<i>New — the draft had no representation of this Sprint 1 field group at all.</i>

Entity	ProfileDisabilityType
Purpose	Multi-select disability/health-condition types, revealed only when the candidate answers Yes to the parent disability question.
Key Attributes	Id, ProfileId (FK), DisabilityType
Primary Key	Id
Foreign Keys	ProfileId → Profile

Business Rules	Populated only if the candidate answers Yes to the disability question. Validated type list (from the live prototype): Visual impairment, Hearing impairment, Speech impairment, Mobility/Physical disability, Chronic illness, Intellectual/Developmental disability, Other. Multi-select — one row per selected type.
Security Class.	HIGH
Change from draft	<i>Confirmed against the live UI: the multi-select-on-Yes design was already correct as originally modeled. Type list is now the validated one shown above rather than an open-ended placeholder.</i>

Entity	ProfileLanguage
Purpose	Multi-select languages spoken.
Key Attributes	Id, ProfileId (FK), Language
Primary Key	Id
Foreign Keys	ProfileId → Profile
Business Rules	One row per language.
Security Class.	LOW
Change from draft	<i>New.</i>

Entity	ProfileHobby
Purpose	Multi-select hobbies/interests.
Key Attributes	Id, ProfileId (FK), Hobby
Primary Key	Id
Foreign Keys	ProfileId → Profile
Business Rules	One row per hobby.
Security Class.	LOW
Change from draft	<i>New.</i>

Entity	Education
Purpose	Education history entries.
Key Attributes	EducationId, ProfileId (FK), Degree, Institution
Primary Key	EducationId
Foreign Keys	ProfileId → Profile
Business Rules	Zero or more per profile.
Security Class.	LOW
Change from draft	<i>Newly catalogued — existed only in the ER image.</i>

Entity	Occupation
Purpose	Occupation history entries.

Key Attributes	OccupationId, ProfileId (FK), JobTitle, Company
Primary Key	OccupationId
Foreign Keys	ProfileId → Profile
Business Rules	Zero or more per profile.
Security Class.	LOW
Change from draft	<i>Newly catalogued.</i>

Entity	Address
Purpose	A profile's two address records: native place (family/ancestral origin) and current residence.
Key Attributes	AddressId, ProfileId (FK), AddressType (Native Current), City, State, Country
Primary Key	AddressId
Foreign Keys	ProfileId → Profile
Business Rules	Exactly two rows per profile — one AddressType=Native (primary) and one AddressType=Current (secondary). Values may be identical if the person has never relocated. Current is the row that gets denormalized onto Profile.Country/State/City for search.
Security Class.	MEDIUM
Change from draft	<i>Resolved (previously Open Issue #2): AddressType added to disambiguate from Profile's own location fields, rather than leaving the two overlapping and undocumented.</i>

Entity	PartnerPreference
Purpose	What a candidate is looking for in a partner (full Part 2 field set).
Key Attributes	PreferenceId, ProfileId (FK, 1:1), PreferredGender (locked), AgeMin, AgeMax, HeightMinCm, HeightMaxCm, PreferredSect, PreferredSubSect, PreferredMadhhab, PreferredSalah, PreferredReligiosity, HijabPref, BeardPref, PreferredMaritalStatus, AcceptChildren, PreferredProfession, EmploymentPref, PreferredEducation, OpenToAnyRegion, PreferredCountry, PreferredCity, PartnerShouldRelocate, LivingArrangement, ChildrenPref, PreferredTimeline, MotherTonguePref, OtherExpectations
Primary Key	PreferenceId
Foreign Keys	ProfileId → Profile
Business Rules	PreferredGender auto-set to the opposite of Profile.Gender and locked — not manually editable. AgeMin/AgeMax, PreferredSect and LivingArrangement mandatory.
Security Class.	LOW
Change from draft	<i>Primary key typo ("preferpartanerId") corrected to PreferenceId; expanded from 8 to the full ~20-field set from FORM_SPEC.md Part 2. WaliInvolvementExpected removed entirely — confirmed not required for this platform; Matching no longer filters or ranks on the matched side's Wali control level.</i>

Entity	QuickMatchPreference
Purpose	Ephemeral answers captured by the pre-registration Quick-Match Wizard — a lightweight teaser flow shown immediately after signup, before the full profile form. Resolves the previously-open question of whether this feature is in scope: confirmed live and built.

Key Attributes	QuickMatchPreferenceId, ProfileId (FK, nullable), WaliFor (Son Daughter, nullable), Country, AgeMin, AgeMax, Sect (multi-select), SalahPref, ReligiosityLevel, MaritalPref (multi-select), Timeline, Language (en hi), CreatedAt
Primary Key	QuickMatchPreferenceId
Foreign Keys	ProfileId → Profile (nullable)
Business Rules	ProfileId is null until the person continues into full registration (“Fill out my full profile”) — up to that point this is anonymous, pre-account data. If the person instead chooses “Skip for now, just browse,” this row is kept for the teaser-match display only and is never reconciled into PartnerPreference. Each of the 8 wizard steps (Wali-for, location, age range, sect, salah, religiosity, marital preference, timeline) can individually be skipped (“Skip — I’ll decide later”), so most attributes are optional in practice.
Security Class.	LOW
Change from draft	<i>New entity, formalizing what was Open Issue #2 (“Quick-Match Wizard status pending”) — confirmed live in the audited UI, with its exact 8-step shape now captured.</i>

Entity	Wali
Purpose	A Wali — any relative (father, mother, uncle, brother, chacha, mamu, appointed Wali, etc.) associated with a candidate. Exactly one Wali slot exists per profile at a time (older, replaced Walis are kept as inactive history rows).
Key Attributes	WaliId, ProfileId (FK, 1:1 active), UserId (FK, nullable), Name, Relation, Phone (nullable), VerificationLevel, ControlLevel, IsActive, CreatedAt
Primary Key	WaliId
Foreign Keys	ProfileId → Profile; UserId → User (nullable)
Business Rules	VerificationLevel ∈ {Verified, Unverified} answers “is this really them” — Verified once the Wali has their own phone-verified account. ControlLevel ∈ {Proxy, None} answers “can they act for the candidate” — Proxy only when the Wali registered the profile directly, or was promoted via an approved WaliChangeRequest. The two are independent: a Wali can become Verified (creates an account) without ever gaining Proxy control, if the candidate never approves the promotion. Only one Wali row per profile has IsActive = true at any time; a Swap (via WaliChangeRequest) deactivates the old row and activates a new one, preserving history. The same UserId may be the active Wali on multiple profiles at once (one Wali managing several candidates, e.g. a father managing more than one daughter’s profile). Relation is free text — the live UI offers Father/Brother/Uncle/Appointed Wali as quick-pick examples, but the field must accept any relative (mother, chacha, mamu, etc.), consistent with the original requirement. Phone is nullable: captured for a Proxy (registered) Wali, but not collected at all for a purely display-only listing in the live UI — a display-only Wali may exist with just a Name and Relation, no contact info on file until/unless he later registers and is promoted.
Security Class.	HIGH
Change from draft	<i>Replaced the single TrustLevel field with two independent fields, VerificationLevel and ControlLevel — collapsing “is this really them” and “can they act for the candidate” into one field broke down once a display-only Wali could later verify their own identity without ever being granted control. Removed ShowContactOnProfile — superseded by the new ContactDisclosure entity, which tracks contact-sharing as an auditable event rather than a static toggle. Phone made explicitly nullable, validated against the live UI’s display-only Wali entry, which never asks for a phone number.</i>

Entity	WaliChangeRequest
Purpose	A request to change who holds the Wali slot on a profile — either promoting an existing display-only Wali to full control, or swapping in a different person entirely. Two different flows share one shape.

Key Attributes	RequestId, ProfileId (FK), RequestType (Promotion Swap), RequestedByUserId (FK), ProposedWaliName, ProposedWaliRelation, ProposedWaliPhone, ProposedWaliUserId (nullable), Reason, Status, ReviewerUserId (FK, nullable), ReviewedAt, CreatedAt
Primary Key	RequestId
Foreign Keys	ProfileId → Profile; RequestedByUserId → User; ProposedWaliUserId → User (nullable); ReviewerUserId → User (nullable)
Business Rules	Promotion: requested by the Wali who wants to move from Unverified/None to Verified/Proxy (their own new account is ProposedWaliUserId); ReviewerUserId must be the candidate (Profile.UserId) — only the candidate can grant control over her own profile. Reason not required for Promotion. Swap: requested by the candidate to replace her current Wali with a different person; Reason is mandatory; ReviewerUserId must have Role = Admin. On approval: Promotion updates the existing Wali row in place (sets UserId, ControlLevel = Proxy, VerificationLevel = Verified); Swap deactivates the current active Wali row (IsActive = false) and inserts a new active row from the Proposed* fields.
Security Class.	MEDIUM
Change from draft	<i>New entity — closes a gap where two structurally different approval flows (candidate-approved promotion vs. Admin-approved swap) would otherwise have needed duplicated, hard-to-maintain logic. One entity, one RequestType field, two reviewer rules.</i>

Entity	ContactDisclosure
Purpose	Append-only audit record of every time a match was actually shown someone's phone number — whose number, why, and who triggered it. This is the single source of truth for “who has seen whose contact info,” given how sensitive that action is.
Key Attributes	DisclosureId, ConversationId (FK), DisclosedToProfileId (FK), DisclosedContactType (WaliContact CandidateContact), DisclosedName, DisclosedPhone, Remark, TriggerType (WaliApproval ManualShareByCandidate NoWaliSelfApproval), TriggeredByUserId (FK), MessageId (FK, nullable), DisclosedAt
Primary Key	DisclosureId
Foreign Keys	ConversationId → Conversation; DisclosedToProfileId → Profile; TriggeredByUserId → User; MessageId → Message (nullable)
Business Rules	Exactly one row per side per conversation, written the moment contact info is actually revealed — never before. DisclosedContactType = WaliContact whenever an active, Proxy-control Wali exists for the disclosing side (TriggerType = WaliApproval), or the candidate manually shares it because her Wali cannot currently log in (TriggerType = ManualShareByCandidate). DisclosedContactType = CandidateContact only when the disclosing side has no active Wali at all (TriggerType = NoWaliSelfApproval). For ManualShareByCandidate, Remark is the candidate's own free-text note (e.g. “my father's account has a login issue, please call him directly”), and MessageId points to the single, one-off system message (Message.MessageType = ContactShareNotice) that actually delivers DisclosedName/DisclosedPhone/Remark to the other side — this is the one narrow exception allowed to be sent while the surrounding conversation otherwise remains Blocked; it does not itself unblock ordinary messaging. DisclosedName/DisclosedPhone are a snapshot at disclosure time — they do not change retroactively if the underlying Wali or candidate later updates their number.
Security Class.	HIGH
Change from draft	<i>Added Remark and MessageId. When a Wali's login is unavailable and the candidate manually shares his contact, the conversation itself stays Blocked (no fallback to the candidate messaging directly) — the only thing that happens is this single, explicitly-tracked one-off system message, so the escape hatch can never quietly turn into an open, unmonitored conversation.</i>

Entity	VerificationRequest
---------------	----------------------------

Purpose	Verification submissions and trust-score history.
Key Attributes	VerificationRequestId, ProfileId (FK), VerificationType, Status, SubmittedAt, ReviewedAt, ReviewedByUserId (FK), Remarks, TrustScore
Primary Key	VerificationRequestId
Foreign Keys	ProfileId → Profile; ReviewedByUserId → User
Business Rules	Multiple verification attempts are allowed. TrustScore updates only after a successful review.
Security Class.	HIGH
Change from draft	<i>ReviewedByUserId now points at User (Role=Admin), closing the previously-undefined "ReviewerId" reference.</i>

Entity	MatchCandidate
Purpose	A directional match: candidates surfaced for a given profile by running its PartnerPreference against the pool.
Key Attributes	MatchId, ProfileId (FK), MatchedProfileId (FK → Profile), CompatibilityScore, TrustScoreAtMatch, BlendedScore, MatchedProfileVerified, Status, GeneratedAt, UpdatedAt
Primary Key	MatchId
Foreign Keys	ProfileId → Profile; MatchedProfileId → Profile
Business Rules	Matching is directional — A's list and B's list are independent rows, potentially with different scores. BlendedScore combines CompatibilityScore with the matched profile's current TrustScore; when VerificationRequest.TrustScore changes, existing rows are updated in place rather than requiring regeneration. Unverified profiles still appear, flagged via MatchedProfileVerified = false.
Security Class.	MEDIUM
Change from draft	<i>Rebuilt from scratch (the draft's entity #9 slot held Wali data by mistake). Score model resolved into three explicit fields instead of one ambiguous "Score" column.</i>

Entity	SubscriptionPlan
Purpose	Defines available subscription tiers.
Key Attributes	PlanId, PlanName, Price, Duration
Primary Key	PlanId
Foreign Keys	—
Business Rules	Price changes do not retroactively affect already-active subscriptions.
Security Class.	LOW
Change from draft	<i>Newly catalogued.</i>

Entity	Subscription
Purpose	A profile's subscription history.
Key Attributes	SubscriptionId, ProfileId (FK), PlanId (FK), Status, StartDate, EndDate, AutoRenew, CreatedAt
Primary Key	SubscriptionId
Foreign Keys	ProfileId → Profile; PlanId → SubscriptionPlan

Business Rules	Only one active subscription per profile at a time. Men's pricing applies the highest single eligible discount (age >55, divorced, widowed, or revert) — discounts never stack; women's accounts are free regardless of these factors.
Security Class.	MEDIUM
Change from draft	<i>No structural change; discount business rule made explicit here rather than living only in prose elsewhere.</i>

Entity	Payment
Purpose	Individual payment transactions against a subscription.
Key Attributes	PaymentId, SubscriptionId (FK), Gateway, GatewayTransactionId, Amount, Currency, Status, PaidAt
Primary Key	PaymentId
Foreign Keys	SubscriptionId → Subscription
Business Rules	GatewayTransactionId must be unique.
Security Class.	MEDIUM
Change from draft	<i>No structural change.</i>

Entity	Conversation
Purpose	A messaging thread between two profiles.
Key Attributes	ConversationId, InitiatorProfileId (FK), ReceiverProfileId (FK), InitiatorManagedByWaliId (FK, nullable), ReceiverManagedByWaliId (FK, nullable), CurrentApprovalStatus, Status, CreatedAt
Primary Key	ConversationId
Foreign Keys	InitiatorProfileId → Profile; ReceiverProfileId → Profile; InitiatorManagedByWaliId → Wali (nullable); ReceiverManagedByWaliId → Wali (nullable)
Business Rules	Starts in PendingApproval. Opens to messaging once both sides' ConversationApproval rows resolve to Approved. For each side, the ManagedByWaliId field is set the moment that side's active, Proxy-control Wali approves — from then on, messages on that side must come from the Wali's account, not the candidate's own login (the candidate is excluded from that specific conversation while the Wali is handling it, even though she retains equal rights everywhere else on her own profile). If a side has no active Proxy Wali, its ManagedByWaliId simply stays null and the candidate messages directly. Blocked users cannot create new conversations.
Security Class.	HIGH
Change from draft	<i>Replaced the single denormalized CurrentApprovalStatus-only model with per-side ManagedByWaliId tracking, so the system knows definitively whether a message on a given side should come from the candidate or from her Wali's account.</i>

Entity	ConversationApproval
Purpose	Append-only log of the approve/revoke decisions that gate a conversation, one row per side per decision.
Key Attributes	ApprovalId, ConversationId (FK), Side (Initiator Receiver), ApprovingUserId (FK), Status, ReviewedAt, Notes
Primary Key	ApprovalId
Foreign Keys	ConversationId → Conversation; ApprovingUserId → User

Business Rules	Never updated in place — a new row is inserted for every decision, preserving full history. ApprovingUserId is always whoever currently controls that side of the conversation: the active Wali's UserId if that profile has an active, Proxy-control Wali, otherwise the profile owner's own UserId (self-approval, no gate). Both sides must independently resolve to Approved before Conversation.CurrentApprovalStatus allows messaging; either side revoking blocks new messages immediately without deleting history.
Security Class.	HIGH
Change from draft	<i>Renamed from WaliApproval and generalized: rather than assuming a Wali always exists, ApprovingUserId now uniformly represents "whoever currently controls this side" — a Proxy Wali if one is active, or the candidate herself if not. This single mechanism now correctly covers proxy-Wali approval, self-approval with no Wali, and (implicitly) blocks any action from a display-only, None-control Wali, without special-casing any of the three.</i>

Entity	Message
Purpose	Individual chat messages within a conversation.
Key Attributes	MessageId, ConversationId (FK), SenderProfileId (FK), SentByUserId (FK), Content, MessageType, SentAt, ReadAt, ModerationFlag
Primary Key	MessageId
Foreign Keys	ConversationId → Conversation; SenderProfileId → Profile; SentByUserId → User
Business Rules	Messages cannot be edited after sending. SenderProfileId identifies which side of the conversation the message represents (unchanged, used for display/grouping); SentByUserId identifies the actual logged-in account that authored it, which must equal either the profile's own UserId or that side's active Wali's UserId per Conversation.*ManagedByWaliId — never anyone else. Ordinary MessageType = Text messages are blocked entirely while a side has no active Proxy Wali approval and no self-approval (i.e. while genuinely Blocked, such as when a Wali's login is unavailable and he has never approved). The one exception is MessageType = ContactShareNotice — a single, system-generated message carrying a Wali's contact info plus the candidate's own remark (see ContactDisclosure), which is allowed even while the conversation is otherwise Blocked, and does not itself unblock ordinary messaging. ModerationFlag is set by automated content moderation — independent of, and not a substitute for, Wali approval.
Security Class.	HIGH
Change from draft	<i>Removed the duplicate ApprovalStatus column (unchanged from v2.1). Added SentByUserId, separate from SenderProfileId, so the system — and any audit/dispute review — can always tell whether a given message was actually typed by the candidate or by her managing Wali, even though the match on the other end only ever sees one counterparty. Added the ContactShareNotice MessageType as the sole narrow exception to the otherwise-total messaging block when a Wali is unavailable.</i>

Entity	Attachment
Purpose	File attachments on a message.
Key Attributes	AttachmentId, MessageId (FK), StorageKey, FileName
Primary Key	AttachmentId
Foreign Keys	MessageId → Message
Business Rules	Accessed only via signed URLs, never a public path.
Security Class.	HIGH

Change from draft	<i>No structural change.</i>
--------------------------	------------------------------

Entity	Photo
Purpose	Profile photos.
Key Attributes	PhotoId, ProfileId (FK), StorageKey, ThumbnailKey, Visibility, ModerationStatus, UploadedAt
Primary Key	PhotoId
Foreign Keys	ProfileId → Profile
Business Rules	Stored privately; accessible only through signed URLs. Requires moderation approval before becoming visible to other users — visibility hiding must be enforced server-side, not just in the UI. Maximum 4 photos per profile (validated against the live prototype).
Security Class.	HIGH
Change from draft	<i>Added the 4-photo maximum, confirmed against the live UI.</i>

Entity	ProfileVideo
Purpose	An optional short video introduction for a profile — either the candidate introducing herself, or her Wali introducing her on her behalf.
Key Attributes	VideoId, ProfileId (FK), IntroducedBy (Candidate Wali), StorageKey, DurationSeconds, ModerationStatus, UploadedAt
Primary Key	VideoId
Foreign Keys	ProfileId → Profile
Business Rules	Optional. Maximum 1 minute duration, MP4/MOV only (validated against the live prototype). Same signed-URL-only access and moderation-gate rules as Photo. At most one active video per profile.
Security Class.	HIGH
Change from draft	<i>New entity — discovered during the live UI audit; not previously modeled at all.</i>

Entity	PrivacySetting
Purpose	Per-profile visibility controls.
Key Attributes	PrivacySettingId, ProfileId (FK, 1:1), PhotoVisibility, ContactVisibility, ProfileVisibility
Primary Key	PrivacySettingId
Foreign Keys	ProfileId → Profile
Business Rules	Defaults are more restrictive for female profiles, consistent with Profile's default-private business rule.
Security Class.	HIGH
Change from draft	<i>Newly catalogued — existed only in the ER image.</i>

Entity	Report
Purpose	A user-filed report against either a profile or a message.

Key Attributes	ReportId, ReporterProfileId (FK), ReportedProfileId (FK, nullable), ReportedMessageId (FK, nullable), Reason, Status, ReviewedByUserId (FK), ReviewedAt, Resolution
Primary Key	ReportId
Foreign Keys	ReporterProfileId → Profile; ReportedProfileId → Profile (nullable); ReportedMessageId → Message (nullable); ReviewedByUserId → User
Business Rules	Exactly one of ReportedProfileId / ReportedMessageId is populated per report. Reviewed only by users with Role ∈ {Admin, Moderator}.
Security Class.	MEDIUM
Change from draft	<i>Newly defined — referenced in the draft's Relationship Matrix but never actually specified.</i>

5. Attribute Classification (Security & Privacy)

Every field flagged Sensitive or Encrypt = Yes needs a concrete storage decision before implementation. v1.2 update: Email/MobileNumber storage, hashing, and encryption are now Supabase Auth's (GoTrue) responsibility, not this application's — the previous open question about a queryable-encryption mechanism for MobileNumber is resolved by removing that responsibility from our own schema entirely.

Entity	Attribute	Mandatory	Sensitive	Encrypt	Mutable
Supabase auth.users	Email / Phone	Yes	Yes	Managed by Supabase Auth, not this schema	Yes
Supabase auth.users	Password/OTP secret	Yes	Critical	Managed by Supabase Auth (GoTrue), not this schema	Yes
User	Role	Yes	Medium	No	Rarely
User	Status	Yes	No	No	Yes
Profile	FirstName / LastName	Yes	Yes	No	Yes
Profile	DateOfBirth	Yes	High	Yes	No
Profile	Gender	Yes	No	No	No
Profile	MaritalStatus	Yes	Medium	No	Yes
Profile	IsRevert	No	Medium	No	Rarely
Profile	Income	No	High	Yes	Yes
Profile	Bio	No	Low	No	Yes
ProfileChildrenDetail	ChildrenDetailsText	No	High	Yes	Yes
Wali	Name / Phone	Conditional	High	Yes (Phone)	Yes
ContactDisclosure	DisclosedPhone / DisclosedName	Yes	High	Yes	No (snapshot, never retroactively changed)
WaliChangeRequest	Reason	Conditional (required for Swap)	Medium	No	No
Address	AddressType / City / State / Country	Yes	Medium	No	Yes
VerificationRequest	TrustScore	Yes	Medium	No	Yes
VerificationRequest	Remarks	No	Medium	No	Yes
MatchCandidate	BlendedScore	Yes	Low	No	Yes
Payment	Amount	Yes	No	No	No
Payment	GatewayTransactionId	Yes	Medium	Yes	No
Message	Content	Yes	High	Yes	No
Message	ReadAt	No	No	No	Yes

Entity	Attribute	Mandatory	Sensitive	Encrypt	Mutable
Photo	StorageKey	Yes	High	No (access-controlled, not encrypted at rest by default)	No
Photo	Visibility	Yes	Medium	No	Yes
Report	Reason / Resolution	Conditional	Medium	No	Yes

6. Relationship Matrix

Parent	Child	Cardinality	Notes
User	Profile	1:1	
User	Supabase auth.users	1:1	Not an app-owned table — external identity managed by Supabase Auth
User	LoginLog	1:N	
User	Wali	1:N (optional)	Only populated when a Wali holds an account
User	VerificationRequest	1:N	as reviewer (ReviewedByUserId)
User	Report	1:N	as reviewer (ReviewedByUserId)
Profile	PrivacySetting	1:1	
Profile	MarriageIntention	1:1	
Profile	ReligiousPreference	1:1	
Profile	ProfileChildrenDetail	1:1 conditional	Divorced/Widowed only
Profile	ProfileDisabilityType	1:N conditional	
Profile	ProfileLanguage	1:N	
Profile	ProfileHobby	1:N	
Profile	Education	1:N	
Profile	Occupation	1:N	
Profile	Address	1:2 (exactly Native + Current)	Resolved — no longer overlaps with Profile's denormalized location fields
Profile	PartnerPreference	1:1	
Profile	Wali	1:1 active	One active Wali slot per profile; older, swapped-out Walis retained as inactive history
Profile	Photo	1:N	Max 4 per profile
Profile	ProfileVideo	0..1	
Profile	QuickMatchPreference	0..1	Nullable until the wizard is linked to a completed registration
Profile	VerificationRequest	1:N	
Profile	Subscription	1:N	
Profile	MatchCandidate	1:N	as seeker (ProfileId)
Profile	MatchCandidate	1:N	as candidate (MatchedProfileId)
Profile	Conversation	1:N	as initiator
Profile	Conversation	1:N	as receiver
Profile	Message	1:N	as sender
Profile	Report	1:N	as reporter
Profile	Report	1:N	as reported target (nullable)
Profile	WaliChangeRequest	1:N	Promotion requests reviewed by this profile's own owner; Swap requests reviewed by Admin

Parent	Child	Cardinality	Notes
ReligiousPreference	ProfileLifestyleGoal	1:N	
ReligiousPreference	ProfileWeddingRitual	1:N conditional	Large wedding only
Wali	ConversationApproval	1:N	Only when this Wali is the ApprovingUserId for a side
Wali	Conversation	1:N	as the managing Wali for a side (Initiator- or ReceiverManagedByWaliId)
Conversation	Message	1:N	
Conversation	ConversationApproval	1:N	One row per approve/revoke decision, per side
Conversation	ContactDisclosure	0..2	At most one disclosure per side, written the moment contact info is revealed
Profile	ContactDisclosure	1:N	as the recipient of a disclosure (DisclosedToProfileId)
Message	Attachment	1:N	
Message	Report	0..1	Optional — a report may target a specific message
SubscriptionPlan	Subscription	1:N	
Subscription	Payment	1:N	

7. Entity-Relationship Diagram

The full ER diagram is on the following landscape page (32 entities across 12 modules, color-coded by owning module). It reflects the v3.4 model — backend on self-hosted Supabase (Auth/Storage/Realtime), business logic in Next.js: Wali carries independent VerificationLevel/ControlLevel fields, WaliChangeRequest models both the candidate-approved promotion and Admin-approved swap flows, ConversationApproval generalizes approval to whoever currently controls each side, and ContactDisclosure audits every reveal of contact information.

Colour key: red = Auth, blue = Profile core, teal = Religious/lifestyle, orange = Profile detail & Wali, purple = Partner Preference, green = Matching, yellow = Subscription/Billing, grey = Reporting.

8. Cross-Cutting Business Rules

- **Profile completeness gate:** a profile must reach $\geq 60\%$ completion (CompletionPercentage, stored and recomputed on save) before it is eligible to appear in Search or Matching.
- **Female privacy default:** female profiles default to private visibility (PrivacySetting.ProfileVisibility); more restrictive photo/contact defaults apply to female profiles.
- **Signed-URL-only media:** Photo and Attachment are never served from a public path — visibility and moderation gates are enforced at the API/serialization layer, not the UI, so a hidden field can never leak in a raw JSON payload.
- **Subscription discounts (men only):** age > 55 (55%), divorced (50%), widowed (50%), revert (50%). Only the single highest-eligible discount applies — discounts never stack. Women's accounts are free regardless of these factors.
- **Wali control model (v3):** exactly one Wali slot per profile. A Wali's VerificationLevel (Verified/Unverified) and ControlLevel (Proxy/None) are tracked independently — a Wali can verify their own identity without ever being granted decision-making control. Only a Proxy-control Wali, or the candidate herself when no Wali is active, can approve/reject conversations; a display-only (None-control) Wali only receives notifications (new match, conversation started, marriage-stage progress) and never gates anything.
- **Wali change requests:** promoting a display-only Wali to full Proxy control is approved by the candidate herself; swapping the Wali for a different person entirely requires a reasoned request approved by Admin. These are structurally different approval flows sharing one WaliChangeRequest entity, distinguished by RequestType.
- **Per-side conversation control and messaging routing:** a Conversation gates independently on each side via ConversationApproval, where the approving party is always whoever currently controls that side (Proxy Wali if active, otherwise the candidate). The moment a side's Proxy Wali approves, that side's messaging is handled exclusively by the Wali's account (Conversation.*ManagedByWaliId) — the candidate is excluded from that specific conversation, even though she retains equal rights everywhere else on her own profile.
- **Contact disclosure is an audited event, not a static flag:** a match is shown the Wali's contact info the moment that side's Proxy Wali approves, or if the candidate manually shares it because her Wali's login is unavailable (via a single system-generated ContactShareNotice message plus her own remark — the surrounding conversation stays Blocked either way, this is a narrow exception, not a fallback that opens messaging); a match is shown the candidate's own contact info only if no Wali is active on that side at all. Every disclosure is recorded in ContactDisclosure with who triggered it and why.
- **Directional, trust-blended matching:** MatchCandidate rows are generated per-profile (A→B and B→A are independent rows/scores). BlendedScore combines PartnerPreference compatibility with the matched profile's live TrustScore; a verification-status change updates existing rows rather than requiring regeneration.
- **Fields never collected** (unchanged from the original product decision): sexual history, exact home address, credit score/debt, social media credentials, political affiliation, forced medical/immigration disclosures, criminal history beyond what's legally required, photos of children, skin colour as a required field, IQ results, menstrual/reproductive medical details. No entity above should ever grow a column for any of these without a deliberate, separately-reviewed product decision.
- **UI implementation guidance — no redundant Wali-role question:** a live prototype audit found a second, confusing “Who is filling out this profile?” question (Myself/Parent-Family/Guardian/Wali — quoting the live prototype's own literal option labels) appearing inside the profile form, in addition to the role already chosen at registration. Future UI builds should not collect this a second time — there are only two real states in this model (a Wali with ControlLevel = Proxy, or no controlling Wali at all), regardless of which relative it is, and that state is already fully determined by registration choice plus Wali.ControlLevel.
- **Confirmed — phone-privacy copy matches the design, doesn't contradict it:** “your number stays private” (shown at registration) refers to User.MobileNumber (login/OTP), which is never shown to other members. It does not contradict ContactDisclosure: a candidate's own number is only ever revealed to a match when no active Wali exists

on her side (TriggerType = NoWaliSelfApproval); whenever an active, registered Wali exists, his contact is what's disclosed instead, exactly as designed.

9. Open Issues & Deferred Scope

Genuinely unresolved items — flagged rather than guessed at, per the review process for this document.

#	Issue	Why it matters	Needs
1	Income bands remain India-Rupee-specific (Below ■20K ... Above ■2L) despite multi-country support. (Note: the live prototype's separate State dropdown, previously flagged as India-only, has since been removed entirely — see row 27 — so this issue now applies only to Income.)	A non-India-resident user sees irrelevant currency bands — acceptable for an India-first launch, but a real gap the moment a non-India user actually registers.	Confirmed as a known limitation, to be addressed in actual implementation — country-aware income bands, not blocking for MVP.